

```
=====
FILE: README.md
=====
```

DDI Dataset Codebase

Code for loading DDI data and the models from our paper:
Disparities in Dermatology AI Performance on a Diverse, Curated Clinical Image Set

For more information, please visit our project page [here](<https://ddi-dataset.github.io/>) and read our paper [here](<https://www.science.org/doi/full/10.1126/sciadv.abq6147>).

Our models can be downloaded [here](https://drive.google.com/drive/folders/1oQ53WH_Tp6rcLZjRp_-UBOQcMl-b1kkP) or through the provided code.

The DDI dataset is hosted on Azure. You get a **time-limited download link** (SAS URL) from the [Stanford AIMI dataset page](<https://stanfordaimi.azurewebsites.net/datasets/35866158-8196-48d8-87bf-50dca81df965>). That link cannot be opened directly in a browser; use one of the methods below to download the entire dataset.

Option 1: AzCopy (command line)

1. **Install AzCopy**

- Windows: [Download AzCopy v10](<https://aka.ms/downloadazcopy-v10-windows>) (zip), extract, and optionally add the folder to your PATH.
- Or install via winget: ``winget install Microsoft.AzCopy``

2. **Download the dataset**

Replace ``YOUR_SAS_URL`` with the full SAS URL from Stanford AIMI (the one that starts with ``https://aimistanforddatasets01.blob...`` and includes ``?sv=...&sig=...``).

Replace ``DDI`` with the folder where you want the data (e.g. ``C:\Data\DDI``).

```
```bash
azcopy copy "YOUR_SAS_URL" "DDI" --recursive
```
```

Example (use your own link; this one expires):

```
```bash
azcopy copy
"https://aimistanforddatasets01.blob.core.windows.net/ddidiversedermatologyimages?sv=2019-02-02&sr=c&sig=..." "DDI" --recursive
```
```

3. **Match the expected layout**

The code expects:

- ``DDI/ddi_metadata.csv``

- `DDI/images/` with all `.png` files (e.g. `000001.png`, `000002.png`, ...)

If the Azure container has a different structure, move the CSV and images into this layout after download.

Option 2: Azure Storage Explorer (GUI)

1. ****Install**** [Azure Storage

Explorer](<https://azure.microsoft.com/products/storage/storage-explorer/>).

2. ****Connect with the SAS URL****

- Open Storage Explorer ****Connect**** (plug icon) ****Blob container or directory**** ****Shared access signature URL (SAS)****.

- Paste the full SAS URL from Stanford AIMI ****Next**** ****Connect****.

3. ****Download****

- Open the container, select the folder or all blobs, then ****Download**** and choose your local folder (e.g. `DDI`).

4. ****Match the expected layout****

Same as above: ensure `ddi\_metadata.csv` and an `images/` folder with all `.png` files are under your `DDI` directory.

> ****Note:**** The SAS link expires (often after a few weeks). If you get authentication or expiry errors, request a new download link from the Stanford AIMI dataset page.

Description

We include code to download and load our models (`ddi\_model.py`), load the DDI dataset (`ddi\_dataset.py`), evaluate our models on the DDI dataset (`eval\_ddi.py`) as well as evaluate our models on an arbitrary dataset (`eval\_data.py`). For `eval\_ddi.py` and `eval\_data.py`, we provide a command line interface with the following arguments:

- `model\_dir`: File path for where to save models.
- `model`: Name of the model to load (HAM10000, DeepDerm, GroupDRO, CORAL, or CDANN).
- `no\_download`: Set to disable downloading models.
- `data\_dir`: Folder containing dataset to load. In `eval\_ddi.py`, `data\_dir` should be the root directory and contain (1) a subfolder called `images` containing all the DDI images and (2) a CSV file called `ddi\_metadata.csv`. In `eval\_data.py`, the structure should match the root directory in [torchvision.datasets.ImageFolder](<https://pytorch.org/vision/stable/datasets.html#torchvision.datasets.ImageFolder>) with 2 classes: benign (class 0) and malignant (class 1).
- `eval\_dir`: Folder to store evaluation results.
- `use\_gpu`: Set to use GPU for evaluation.
- `plot`: Set to show ROC plot.

Example usage

- Evaluate `DeepDerm` model on the DDI dataset. Data (not included in this repo) is stored in the `DDI` directory, and results will be saved in the `DDI-results` directory.

```
```bash
```

```
>>>python3 eval_ddi.py --model=DeepDerm --data_dir=DDI --eval_dir=DDI-results
```

```
```
```

- Evaluate `DeepDerm` model on your own dataset (must be annotated as benign/malignant). Data (not included in this repo) is stored in the `MyData` directory, and results will be saved in the `DDI-results` directory.

```
```bash
```

```
>>>python3 eval_data.py --model=DeepDerm --data_dir=MyData --eval_dir=DDI-results
```

```
```
```

Citation

If you find this code useful or use the DDI dataset in your research, please cite:

```
```
```

```
@article{daneshjou2022disparities,
 title={Disparities in dermatology AI performance on a diverse, curated clinical
image set},
 author={Daneshjou, Roxana and Vodrahalli, Kailas and Novoa, Roberto A and Jenkins,
Melissa and Liang, Weixin and Rotemberg, Veronica and Ko, Justin and Swetter, Susan
M and Bailey, Elizabeth E and Gevaert, Olivier and others},
 journal={Science advances},
 volume={8},
 number={31},
 pages={eabq6147},
 year={2022},
 publisher={American Association for the Advancement of Science}
}
```

```
```
```

```
=====
```

FILE: auc_all_preprocessed.py

```
=====
```

```
import os  
import pickle  
import subprocess
```

```
# -----
```

```
# Configuration
```

```
# -----
```

```
preprocessed_dirs = [  
    #"images_bilateral_optimized",  
    #"images_bilateral+msrcr_optimized",  
    #"images_non_local_means+msrcr_optimized",  
    #"images_bilateral+illumination_comp_optimized",
```

```

    #"images_z_score_norm+adaptive_gamma_optimized",
    #"images_z_score_norm+percentile_norm_optimized",
    #"images_bilateral+z_score_norm_optimized",
    "images_msrrcr+local_contrast_optimized",
    "images_clahe+adaptive_gamma_optimized",
    "images_non_local_means+msrrcr_optimized",
    "images_non_local_means+msrrcr+adaptive_gamma_optimized",
    "images_non_local_means+illumination_comp+clahe_optimized",
    #"images_illumination_comp_optimized",
    #"images_illumination_comp+local_contrast_optimized",
    #"images_illumination_comp+adaptive_gamma_optimized"
]

models = ["DeepDerm"]
# , "HAM10000"
eval_base_dir = "DDI-results"
meta_file = "DDI/ddi_metadata.csv"

# Replace "python" with your venv Python path
venv_python = r"C:\Users\LeoYa\GitHub\DDI-Code\.venv\Scripts\python.exe"

for model in models:
    for preproc in preprocessed_dirs:
        eval_file = f"DDI-results_{preproc}_{model}/{model}-evaluation.pkl"
        if os.path.exists(eval_file):
            print(f"\n=== {model} | {preproc} ===")
            cmd = [
                venv_python, "auc_by_skin_tone.py",
                "--preds", eval_file,
                "--meta", meta_file
            ]
            subprocess.run(cmd)
        else:
            print(f"Missing eval file: {eval_file}")

=====
FILE: auc_by_skin_tone.py
=====
import argparse
import pandas as pd
import numpy as np
import os
from sklearn.metrics import roc_auc_score

def auc(df: pd.DataFrame) -> float:
    if df["y_true"].nunique() < 2:
        return float("nan")
    return roc_auc_score(df["y_true"], df["y_score"])

```

```

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--preds", required=True, help="CSV with
DDI_file,y_true,y_score")
    ap.add_argument("--meta", required=True, help="ddi_metadata.csv")
    ap.add_argument("--save_dir", help="Directory to save skin tone specific
dataframes as pickle files")
    args = ap.parse_args()

    if args.preds.endswith(".pkl"):
        data = pd.read_pickle(args.preds)
        # Extract y_score and DDI_file from pickle dict
        # Assuming predicted_labels is (N,) scores and images is list of paths
        y_score = data["predicted_labels"]
        # Convert full paths to filenames to match metadata
        ddi_files = [os.path.basename(p) for p in data["images"]]

        preds = pd.DataFrame({
            "DDI_file": ddi_files,
            "y_score": y_score
        })
    else:
        preds = pd.read_csv(args.preds)
        meta = pd.read_csv(args.meta)

    # Normalize target
    meta["y_true"] = meta["malignant"].astype(str).str.lower().map({"true": 1,
"false": 0})
    if meta["y_true"].isna().any():
        raise ValueError("malignant column must be True/False (or true/false).")

    # Join on filename
    df = preds.merge(
        meta[["DDI_file", "skin_tone", "y_true"]],
        on="DDI_file",
        how="inner",
        suffixes=("", "_meta"),
    )

    # Use metadata truth to avoid accidental mismatch
    if "y_true_meta" in df.columns:
        df["y_true"] = df["y_true_meta"]
        df = df.drop(columns=["y_true_meta"])

    # Ensure types
    df["skin_tone"] = pd.to_numeric(df["skin_tone"], errors="coerce")
    df["y_score"] = pd.to_numeric(df["y_score"], errors="coerce")
    df = df.dropna(subset=["skin_tone", "y_score", "y_true"])

    overall = auc(df)

```

```

g12 = auc(df[df["skin_tone"] == 12])
g56 = auc(df[df["skin_tone"] == 56])
g34 = auc(df[df["skin_tone"] == 34])

print(f"Overall AUC: {overall:.4f}")
print(f"Skin tone 12 AUC: {g12:.4f} (n={len(df[df['skin_tone']==12])})")
print(f"Skin tone 34 AUC: {g34:.4f} (n={len(df[df['skin_tone']==34])})")
print(f"Skin tone 56 AUC: {g56:.4f} (n={len(df[df['skin_tone']==56])})")

if not np.isnan(g12) and not np.isnan(g56):
    print(f"Gap (12 - 56): {(g12 - g56):.4f}")

if args.save_dir:
    os.makedirs(args.save_dir, exist_ok=True)
    df[df["skin_tone"] == 12].to_pickle(os.path.join(args.save_dir,
fst_12.pkl"))
    df[df["skin_tone"] == 34].to_pickle(os.path.join(args.save_dir,
fst_34.pkl"))
    df[df["skin_tone"] == 56].to_pickle(os.path.join(args.save_dir,
fst_56.pkl"))
    print(f"Saved skin tone splits to {args.save_dir}")

if __name__ == "__main__":
    main()

```

```

=====
FILE: bayes_opt_efficiency.py
=====
"""

```

```

Bayesian Optimization for High-Impact Preprocessing Combinations
Only methods/combinations with >20% gap improvement in HAM10000 or DeepDerm
Requires: pip install optuna opencv-python
"""

import os
import glob
import shutil
import tempfile
from multiprocessing import cpu_count
from multiprocessing.pool import ThreadPool
import optuna
import cv2
import numpy as np
import pandas as pd
import torch
from generate_preprocessed import method_map, process_image, process_image_pair,
process_image_triple
from eval_ddi import load_model, DDI_Dataset, eval_model
from PIL import Image

```

```

# -----
# Preprocessing methods and parameter ranges
#
# Only methods whose baseline functions actually accept a tunable parameter
# are listed here with their correct parameter names matching the function
# signatures in generate_preprocessed.py.
#
# Fixed methods with no tunable parameter (white_balance, adaptive_gamma, msrccr)
# can still appear in combos but will use their default behaviour.
# -----

preprocessing_methods = {
    "clahe": {"clip_limit": (0.5, 4.0)}, # apply_clahe(clip_limit)
    "illumination_comp": {"sigma": (10.0, 100.0)}, #
    apply_illumination_comp(sigma)
    "percentile_norm": {"range_": (80.0, 99.0)}, #
    apply_percentile_norm(range_)
    "local_contrast": {"strength": (0.1, 2.0)}, #
    apply_local_contrast(strength)
    "bilateral": {"sigma_color": (10.0, 150.0)}, #
    apply_bilateral_filter(sigma_color)
    "non_local_means": {"h": (5, 15)}, # apply_non_local_means(h)
}

# Slow methods â€” reduce n_trials automatically for combos containing these
SLOW_METHODS = {}
N_TRIALS_DEFAULT = 20
N_TRIALS_SLOW = 10

# -----
# Only methods/combinations with >20% gap reduction
# Use "+" as separator between methods (method names contain underscores)
# -----
high_impact_combos = [
    # Single methods
    # "illumination_comp",
    # "percentile_norm",
    # "local_contrast",
    # "bilateral",
    # "clahe",

    # Combinations â€” use + as separator
    # "illumination_comp+local_contrast",
    # "illumination_comp+adaptive_gamma",
    # "illumination_comp+clahe",
    # "clahe+adaptive_gamma",
    # "non_local_means+illumination_comp",
    "Z_score_norm+percentile_norm",
    "bilateral+Z_score_norm",

```

```

        "bilateral+msrcr+Z_score_norm",
        # "non_local_means+msrcr+adaptive_gamma",
        # "non_local_means+illumination_comp+clahe",
        # "non_local_means+illumination_comp+percentile_norm",
    ]

# Detect GPU once at startup
USE_GPU = torch.cuda.is_available()
print(f"GPU available: {USE_GPU}")
N_CORES = cpu_count()
print(f"CPU cores available: {N_CORES}")

# -----
# Picklable callable class
# -----
class PreprocessTransform:
    """
    Applies a sequence of preprocessing steps to an image then runs
    test_transform. Defined at module level so it can be pickled by
    multiprocessing workers.

    steps: list of (method_name, param_value) tuples.
           param_value is None for fixed methods.
    """
    def __init__(self, steps: list):
        self.steps = steps

    def _apply_method(self, cv_img, method_name, param_value):
        if method_name == "clahe":
            return method_map["clahe"](cv_img, clip_limit=param_value)
        elif method_name == "illumination_comp":
            return method_map["illumination_comp"](cv_img, sigma=param_value)
        elif method_name == "percentile_norm":
            return method_map["percentile_norm"](cv_img, range_=param_value)
        elif method_name == "local_contrast":
            return method_map["local_contrast"](cv_img, strength=param_value)
        elif method_name == "bilateral":
            return method_map["bilateral"](cv_img, sigma_color=param_value)
        elif method_name == "non_local_means":
            return method_map["non_local_means"](cv_img, h=param_value)
        else:
            # Fixed methods (white_balance, adaptive_gamma, msrcr, etc.)
            return method_map.get(method_name, lambda x: x)(cv_img)

    def __call__(self, img):
        from ddi_dataset import test_transform
        cv_img = cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)
        for method_name, param_value in self.steps:
            cv_img = self._apply_method(cv_img, method_name, param_value)

```

```

        pil_img = Image.fromarray(cv2.cvtColor(cv_img, cv2.COLOR_BGR2RGB))
        return test_transform(pil_img)

# -----
# Evaluate preprocessed dataset and extract skin-tone AUCs
# -----
def run_evaluation(dataset, model, num_workers=4):
    results = eval_model(model, dataset, use_gpu=USE_GPU, show_plot=False,
                          num_workers=num_workers)

    df = pd.DataFrame({
        "DDI_file": [os.path.basename(p) for p in results["images"]],
        "y_score": results["predicted_labels"]
    })
    meta = pd.read_csv(os.path.join("DDI", "ddi_metadata.csv"))
    meta["y_true"] = meta["malignant"].astype(str).str.lower().map({"true": 1,
"false": 0})

    df = df.merge(meta[["DDI_file", "skin_tone", "y_true"]], on="DDI_file",
how="inner")
    df["skin_tone"] = pd.to_numeric(df["skin_tone"], errors="coerce")
    df["y_score"] = pd.to_numeric(df["y_score"], errors="coerce")
    df = df.dropna(subset=["skin_tone", "y_score", "y_true"])

    def auc_sub(df_sub):
        if df_sub["y_true"].nunique() < 2:
            return np.nan
        from sklearn.metrics import roc_auc_score
        return roc_auc_score(df_sub["y_true"], df_sub["y_score"])

    auc_12 = auc_sub(df[df["skin_tone"] == 12])
    auc_56 = auc_sub(df[df["skin_tone"] == 56])
    overall_auc = auc_sub(df)

    return auc_12, auc_56, overall_auc

# -----
# Objective function generator
# -----
def make_objective(combo, model):
    split_methods = combo.split("+")

    def objective(trial):
        trial_params = {}
        for m in split_methods:
            if m not in preprocessing_methods:
                continue
            for param, (low, high) in preprocessing_methods[m].items():

```

```

        key = f"{m}_{param}"
        if isinstance(low, int) and isinstance(high, int):
            trial_params[key] = trial.suggest_int(key, low, high)
        else:
            trial_params[key] = trial.suggest_float(key, low, high)

# Build steps list
steps = []
for m in split_methods:
    if m not in preprocessing_methods:
        steps.append((m, None))
        continue
    param_key = list(preprocessing_methods[m].keys())[0]
    param_value = trial_params[f"{m}_{param_key}"]
    steps.append((m, param_value))

tmp_dir = tempfile.mkdtemp()
tmp_img_dir = os.path.join(tmp_dir, "images")
os.makedirs(tmp_img_dir)

try:
    helper = PreprocessTransform(steps)
    image_paths = glob.glob(os.path.join("DDI", "images", "*.png"))

    if len(image_paths) == 0:
        print(" ERROR: No images found in DDI/images/ â€” check path")
        return float("inf")

# Parallel preprocessing using ThreadPool
def preprocess_one(img_path):
    filename = os.path.basename(img_path)
    cv_img = cv2.imread(img_path)
    if cv_img is None:
        print(f" Failed to READ: {img_path}")
        return
    for method_name, param_value in steps:
        cv_img = helper._apply_method(cv_img, method_name, param_value)
    cv2.imwrite(os.path.join(tmp_img_dir, filename), cv_img)

with ThreadPool(processes=N_CORES) as pool:
    pool.map(preprocess_one, image_paths)

from ddi_dataset import test_transform
dataset = DDI_Dataset(
    tmp_dir,
    img_dirname="images",
    csv_path=os.path.join("DDI", "ddi_metadata.csv"),
    transform=test_transform,
)

```

```

    auc_12, auc_56, overall_auc = run_evaluation(dataset, model)

    gap      = auc_12 - auc_56
    penalty = max(0, 0.54 - overall_auc) * 2

    print(f"  params={trial_params}, auc_12={auc_12:.4f},
auc_56={auc_56:.4f}, "
          f"overall={overall_auc:.4f}, gap={gap:.4f}")

    return gap + penalty

finally:
    shutil.rmtree(tmp_dir, ignore_errors=True)

return objective

# -----
# Run Bayesian Optimization
# -----
if __name__ == "__main__":
    model_name = "HAM10000" # or "DeepDerm"
    weights_path = f"DDI-models/{model_name.lower()}.pth"
    results = []

    print("Loading model for evaluation...")
    model = load_model(model_name, weights_path=weights_path, save_dir="DDI-models",
download=False)

    for combo in high_impact_combos:
        # Use fewer trials for combos containing slow methods
        split_methods = combo.split("+")
        n_trials = N_TRIALS_SLOW if any(m in SLOW_METHODS for m in split_methods)
    else N_TRIALS_DEFAULT

    print(f"\n=== Optimizing: {combo} (n_trials={n_trials}) ===")
    study = optuna.create_study(direction='minimize')
    study.optimize(make_objective(combo, model), n_trials=n_trials)
    print("Best params:", study.best_params)
    print(f"Best gap: {study.best_value:.4f}")
    results.append({"combo": combo, "best_params": study.best_params,
"best_gap": study.best_value})

    # Save best params for this combo to CSV immediately after optimization
    csv_out_path = "optimization_results.csv"
    row = {"combo": combo, "best_gap": study.best_value}
    row.update(study.best_params)
    row_df = pd.DataFrame([row])
    if os.path.exists(csv_out_path):
        row_df.to_csv(csv_out_path, mode="a", header=False, index=False)

```

```

else:
    row_df.to_csv(csv_out_path, mode="w", header=True, index=False)
    print(f"Best params appended to {csv_out_path}")

# Save optimized images to disk
print("Saving optimized images...")
best_params = study.best_params
img_out_dir = os.path.join("DDI", f"images_{combo}_optimized")
os.makedirs(img_out_dir, exist_ok=True)

image_paths = glob.glob(os.path.join("DDI", "images", "*.png"))
best_steps = []
for m in split_methods:
    if m not in preprocessing_methods:
        best_steps.append((m, None))
        continue
    param_key = list(preprocessing_methods[m].keys())[0]
    param_value = best_params[f"{m}_{param_key}"]
    best_steps.append((m, param_value))

helper = PreprocessTransform(best_steps)

def save_image(img_path):
    filename = os.path.basename(img_path)
    try:
        cv_img = cv2.imread(img_path)
        if cv_img is None:
            return f"Failed to read {img_path}"
        for method_name, param_value in best_steps:
            cv_img = helper._apply_method(cv_img, method_name, param_value)
        cv2.imwrite(os.path.join(img_out_dir, filename), cv_img)
        return None
    except Exception as e:
        return f"{filename}: {str(e)}"

with ThreadPool(processes=N_CORES) as pool:
    errors = list(pool.imap_unordered(save_image, image_paths))

errors = [e for e in errors if e is not None]
if errors:
    print(f"{len(errors)} images failed to save:")
    for e in errors[:10]:
        print(e)

# Sort and print final results
results = sorted(results, key=lambda x: x["best_gap"])
print("\n=== Top Combinations ===")
for r in results:
    print(f"Combo: {r['combo']}, Best Gap: {r['best_gap']:.4f}, Params: {r['best_params']}")

```

```

=====
FILE: bayes_opt_high_improvement.py
=====
"""
Bayesian Optimization for High-Impact Preprocessing Combinations
Only methods/combinations with >20% gap improvement in HAM10000 or DeepDerm
Requires: pip install optuna opencv-python
"""

import os
import glob
import shutil
import tempfile
from multiprocessing import cpu_count
import optuna
import cv2
import numpy as np
import pandas as pd
from generate_preprocessed import method_map, process_image, process_image_pair,
process_image_triple
from eval_ddi import load_model, DDI_Dataset, eval_model
from PIL import Image

# -----
# Preprocessing methods and parameter ranges
#
# Only methods whose baseline functions actually accept a tunable parameter
# are listed here with their correct parameter names matching the function
# signatures in generate_preprocessed.py.
#
# Fixed methods with no tunable parameter (white_balance, adaptive_gamma)
# can still appear in combos but will use their default behaviour.
# -----

preprocessing_methods = {
    "clahe": {"clip_limit": (0.5, 4.0)}, #
    apply_clahe(clip_limit)
    "illumination_comp": {"sigma": (10.0, 100.0)}, #
    apply_illumination_comp(sigma)
    "percentile_norm": {"range_": (80.0, 99.0)}, #
    apply_percentile_norm(range_)
    "local_contrast": {"strength": (0.1, 2.0)}, #
    apply_local_contrast(strength)
    "bilateral": {"sigma_color": (10.0, 150.0)}, #
    apply_bilateral_filter(sigma_color)
    "non_local_means": {"h": (5, 15)}, #
    apply_non_local_means(h)
}

```

```

# -----
# Only methods/combinations with >20% gap reduction
# Use "+" as separator between methods (method names contain underscores)
# -----
high_impact_combos = [
    # Single methods
    # "illumination_comp",
    # "percentile_norm",
    # "local_contrast",
    # "bilateral",
    # "clahe",

    # Combinations " use + as separator
    # "illumination_comp+local_contrast",
    # "illumination_comp+adaptive_gamma",

    # "illumination_comp+clahe",

    # "non_local_means+illumination_comp",
    # "bilateral+msrcr",
    # "msrcr+local_contrast",
    # "clahe+adaptive_gamma",
    # "non_local_means+msrcr",
    # "non_local_means+msrcr+adaptive_gamma",
    # "non_local_means+illumination_comp+clahe",
    # "bilateral+illumination_comp",
    # "Z_score_norm+percentile_norm",
    # "bilateral+Z_score_norm",
    # "bilateral+msrcr+Z_score_norm",

    # "non_local_means+illumination_comp+percentile_norm",
]

# -----
# Picklable callable class
# -----
class PreprocessTransform:
    """
    Applies a sequence of preprocessing steps to an image then runs
    test_transform. Defined at module level so it can be pickled by
    multiprocessing workers.

    steps: list of (method_name, param_value) tuples.
           param_value is None for fixed methods.
    """
    def __init__(self, steps: list):
        self.steps = steps

```

```

def _apply_method(self, cv_img, method_name, param_value):
    if method_name == "clahe":
        return method_map["clahe"](cv_img, clip_limit=param_value)
    elif method_name == "illumination_comp":
        return method_map["illumination_comp"](cv_img, sigma=param_value)
    elif method_name == "percentile_norm":
        return method_map["percentile_norm"](cv_img, range_=param_value)
    elif method_name == "local_contrast":
        return method_map["local_contrast"](cv_img, strength=param_value)
    elif method_name == "bilateral":
        return method_map["bilateral"](cv_img, sigma_color=param_value)
    elif method_name == "non_local_means":
        return method_map["non_local_means"](cv_img, h=param_value)
    else:
        # Fixed methods (white_balance, adaptive_gamma, msrccr, etc.)
        # called with no extra arguments, using their defaults
        return method_map.get(method_name, lambda x: x)(cv_img)

```

```

def __call__(self, img):
    from ddi_dataset import test_transform
    cv_img = cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)
    for method_name, param_value in self.steps:
        cv_img = self._apply_method(cv_img, method_name, param_value)
    pil_img = Image.fromarray(cv2.cvtColor(cv_img, cv2.COLOR_BGR2RGB))
    return test_transform(pil_img)

```

```

# -----

```

```

# Evaluate preprocessed dataset and extract skin-tone AUCs

```

```

# -----

```

```

def run_evaluation(dataset, model, num_workers=0):
    import torch
    use_gpu = torch.cuda.is_available()
    results = eval_model(model, dataset, use_gpu=use_gpu, show_plot=False,
num_workers=num_workers)

```

```

    df = pd.DataFrame({
        "DDI_file": [os.path.basename(p) for p in results["images"]],
        "y_score": results["predicted_labels"]
    })
    meta = pd.read_csv(os.path.join("DDI", "ddi_metadata.csv"))
    meta["y_true"] = meta["malignant"].astype(str).str.lower().map({"true": 1,
"false": 0})

```

```

    df = df.merge(meta[["DDI_file", "skin_tone", "y_true"]], on="DDI_file",
how="inner")
    df["skin_tone"] = pd.to_numeric(df["skin_tone"], errors="coerce")
    df["y_score"] = pd.to_numeric(df["y_score"], errors="coerce")
    df = df.dropna(subset=["skin_tone", "y_score", "y_true"])

```

```

def auc_sub(df_sub):
    if df_sub["y_true"].nunique() < 2:
        return np.nan
    from sklearn.metrics import roc_auc_score
    return roc_auc_score(df_sub["y_true"], df_sub["y_score"])

auc_12      = auc_sub(df[df["skin_tone"] == 12])
auc_56      = auc_sub(df[df["skin_tone"] == 56])
overall_auc = auc_sub(df)

return auc_12, auc_56, overall_auc

```

```

# -----
# Objective function generator
# -----
def make_objective(combo, model):
    split_methods = combo.split("+")

    def objective(trial):
        trial_params = {}
        for m in split_methods:
            if m not in preprocessing_methods:
                continue # fixed method "â€”" no param to suggest
            for param, (low, high) in preprocessing_methods[m].items():
                key = f"{m}_{param}"
                if isinstance(low, int) and isinstance(high, int):
                    trial_params[key] = trial.suggest_int(key, low, high)
                else:
                    trial_params[key] = trial.suggest_float(key, low, high)

        # Build steps list
        steps = []
        for m in split_methods:
            if m not in preprocessing_methods:
                steps.append((m, None)) # fixed method, no param
                continue
            param_key = list(preprocessing_methods[m].keys())[0]
            param_value = trial_params[f"{m}_{param_key}"]
            steps.append((m, param_value))

        # Write preprocessed images to disk (matches baseline pipeline exactly)
        tmp_dir = tempfile.mkdtemp()
        tmp_img_dir = os.path.join(tmp_dir, "images")
        os.makedirs(tmp_img_dir)

        helper = PreprocessTransform(steps)
        image_paths = glob.glob(os.path.join("DDI", "images", "*.png"))

        print(f" Found {len(image_paths)} images to preprocess")

```

```

if len(image_paths) == 0:
    print(" ERROR: No images found in DDI/images/ â€” check path")
    shutil.rmtree(tmp_dir)
    return float("inf")

for img_path in image_paths:
    filename = os.path.basename(img_path)
    cv_img = cv2.imread(img_path)
    if cv_img is None:
        print(f" Failed to READ: {img_path}")
        continue
    for method_name, param_value in steps:
        cv_img = helper._apply_method(cv_img, method_name, param_value)
    success = cv2.imwrite(os.path.join(tmp_img_dir, filename), cv_img)
    if not success:
        print(f" Failed to WRITE: {os.path.join(tmp_img_dir, filename)}")

from ddi_dataset import test_transform
dataset = DDI_Dataset(
    tmp_dir,
    img_dirname="images",
    csv_path=os.path.join("DDI", "ddi_metadata.csv"),
    transform=test_transform,
)

auc_12, auc_56, overall_auc = run_evaluation(dataset, model, num_workers=0)
shutil.rmtree(tmp_dir)

gap = auc_12 - auc_56
penalty = max(0, 0.50 - overall_auc) * 2

print(f" params={trial_params}, auc_12={auc_12:.4f}, auc_56={auc_56:.4f}, "
      f"overall={overall_auc:.4f}, gap={gap:.4f}")

return gap + penalty

return objective

```

```

# -----
# Run Bayesian Optimization
# -----

```

```

if __name__ == "__main__":
    model_name = "DeepDerm" # or "DeepDerm"
    weights_path = f"DDI-models/{model_name.lower()}.pth"
    results = []

    print("Loading model for evaluation...")
    model = load_model(model_name, weights_path=weights_path, save_dir="DDI-models",
download=False)

```

```

for combo in high_impact_combos:
    print(f"\n=== Optimizing: {combo} ===")
    study = optuna.create_study(direction='minimize')
    study.optimize(make_objective(combo, model), n_trials=20)
    print("Best params:", study.best_params)
    print(f"Best gap: {study.best_value:.4f}")
    results.append({"combo": combo, "best_params": study.best_params,
"best_gap": study.best_value})

    # Save best params for this combo to CSV immediately after optimization
    csv_out_path = "optimization_results.csv"
    row = {"combo": combo, "best_gap": study.best_value}
    row.update(study.best_params)
    row_df = pd.DataFrame([row])
    if os.path.exists(csv_out_path):
        row_df.to_csv(csv_out_path, mode="a", header=False, index=False)
    else:
        row_df.to_csv(csv_out_path, mode="w", header=True, index=False)
    print(f"Best params appended to {csv_out_path}")

    # Save optimized images to disk
    print("Saving optimized images...")
    best_params = study.best_params
    img_out_dir = os.path.join("DDI", f"images_{combo}_optimized")
    os.makedirs(img_out_dir, exist_ok=True)

    image_paths = glob.glob(os.path.join("DDI", "images", "*.png"))
    split_methods = combo.split("+")
    best_steps = []
    for m in split_methods:
        if m not in preprocessing_methods:
            best_steps.append((m, None))
            continue
        param_key = list(preprocessing_methods[m].keys())[0]
        param_value = best_params[f"{m}_{param_key}"]
        best_steps.append((m, param_value))

    helper = PreprocessTransform(best_steps)

    def save_image(img_path):
        filename = os.path.basename(img_path)
        try:
            cv_img = cv2.imread(img_path)
            if cv_img is None:
                return f"Failed to read {img_path}"
            for method_name, param_value in best_steps:
                cv_img = helper._apply_method(cv_img, method_name, param_value)
            cv2.imwrite(os.path.join(img_out_dir, filename), cv_img)
            return None

```

```

        except Exception as e:
            return f"{filename}: {str(e)}"

from multiprocessing.pool import ThreadPool
n_cores = cpu_count()
with ThreadPool(processes=n_cores) as pool:
    errors = list(pool.imap_unordered(save_image, image_paths))

errors = [e for e in errors if e is not None]
if errors:
    print(f"{len(errors)} images failed to save:")
    for e in errors[:10]:
        print(e)

# Sort and print final results
results = sorted(results, key=lambda x: x["best_gap"])
print("\n=== Top Combinations ===")
for r in results:
    print(f"Combo: {r['combo']}, Best Gap: {r['best_gap']:.4f}, Params: {r['best_params']}")

```

```
=====
```

```
FILE: ddi_dataset.py
```

```
=====
```

```
"""Code for loading DDI Dataset."""
```

```

from torch.utils.data import Subset
from torchvision.datasets import ImageFolder
from torchvision import transforms as T
import os
import pandas as pd
import numpy as np
import cv2
from PIL import Image

```

```
means = [0.485, 0.456, 0.406]
```

```
stds = [0.229, 0.224, 0.225]
```

```
def clahe_on_luminance(pil_img, clip_limit=2.0, tile_grid_size=(8, 8)):
```

```
    """Apply CLAHE to the L channel in LAB space; keep color unchanged.
```

```
    Args:
```

```
        pil_img: PIL Image (RGB).
```

```
        clip_limit: CLAHE contrast limit (higher = more contrast).
```

```
        tile_grid_size: Grid size for adaptive equalization.
```

```
    Returns:
```

```
        PIL Image (RGB) with contrast-enhanced luminance.
```

```
    """
```

```
    img = np.array(pil_img)
```

```

if img.ndim == 2:
    img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
elif img.shape[-1] == 3:
    img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)
l = clahe.apply(l)
lab = cv2.merge([l, a, b])
bgr = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
return Image.fromarray(rgb)

```

```

def build_transform(use_clahe=False, clahe_clip_limit=2.0, clahe_tile_grid_size=(8,
8)):

```

```

    """Build the standard DDI image transform, optionally with CLAHE
preprocessing."""

```

```

    steps = [
        lambda x: x.convert('RGB'),
    ]
    if use_clahe:
        steps.append(lambda x: clahe_on_luminance(x, clip_limit=clahe_clip_limit,
tile_grid_size=clahe_tile_grid_size))
    steps.extend([
        T.Resize(299),
        T.CenterCrop(299),
        T.ToTensor(),
        T.Normalize(mean=means, std=stds),
    ])
    return T.Compose(steps)

```

```

# Default transform (no CLAHE) for backward compatibility
test_transform = build_transform(use_clahe=False)

```

```

# Transform with CLAHE preprocessing (e.g. for training or ablation)
test_transform_clahe = build_transform(use_clahe=True)

```

```

class DDI_Dataset(ImageFolder):
    _DDI_download_link =
"https://stanfordaimi.azurewebsites.net/datasets/35866158-8196-48d8-87bf-50dca81df96
5"

```

```

    """DDI Dataset.

```

```

    Note: assumes DDI data is organized as

```

```

        ./DDI
          /images
            /000001.png

```

```
    /000002.png
    ...
    /ddi_metadata.csv
```

(After downloading from the Stanford AIMI repository, this requires moving all .png files into a new subdirectory titled "images".)

```
Args:
    root      (str): Root directory of dataset.
    csv_path  (str): Path to the metadata CSV file. Defaults to
`{root}/ddi_metadata.csv`
    transform      : Function to transform and collate image input. (can use
test_transform from this file)
"""

def __init__(self, root, img_dirname="images", csv_path=None, download=True,
transform=None, *args, **kwargs):
    self.img_dirname = img_dirname
    if csv_path is None:
        csv_path = os.path.join(root, "ddi_metadata.csv")
    if not os.path.exists(csv_path) and download:
        raise Exception(f"Please visit <{DDI_Dataset._DDI_download_link}> to
download the DDI dataset.")
    assert os.path.exists(csv_path), f"Path not found <{csv_path}>."
    super(DDI_Dataset, self).__init__(root, *args, transform=transform,
**kwargs)
    self.annotations = pd.read_csv(csv_path)
    m_key = 'malignant'
    if m_key not in self.annotations:
        self.annotations[m_key] =
self.annotations['malignancy(malig=1)'].apply(lambda x: x==1)

def find_classes(self, directory):
    """Override to only find the specific image directory."""
    path = os.path.join(directory, self.img_dirname)
    if not os.path.isdir(path):
        raise FileNotFoundError(f"Image directory not found: {path}")
    return [self.img_dirname], {self.img_dirname: 0}

def __getitem__(self, index):
    img, target = super(DDI_Dataset, self).__getitem__(index)
    path = self.imgs[index][0]
    # use first matching row if multiple rows share the same DDI_file (e.g.
duplicate filenames)
    filename = os.path.basename(path)
    match = self.annotations[self.annotations.DDI_file == filename]
    if len(match) == 0:
        raise KeyError(f"No annotation for DDI_file: {filename}")
    row = match.iloc[0]
    target = int(row['malignant']) # 1 if malignant, 0 if benign
    skin_tone = int(row['skin_tone']) # Fitzpatrick 12, 34, or 56
```

```

        return path, img, target, skin_tone

    """Return a subset of the DDI dataset based on skin tones and malignancy of
    lesion.

    Args:
        skin_tone    (list of int): Which skin tones to include in the subset.
        Options are {12, 34, 56}.
        diagnosis    (list of str): Include malignant and/or benign images. Options
        are {"benign", "malignant"}
    """
    def subset(self, skin_tone=None, diagnosis=None):
        skin_tone = [12, 34, 56] if skin_tone is None else skin_tone
        diagnosis = ["benign", "malignant"] if diagnosis is None else diagnosis
        for si in skin_tone:
            assert si in [12,34,56], f"{si} is not a valid skin tone"
        for di in diagnosis:
            assert di in ["benign", "malignant"], f"{di} is not a valid diagnosis"
        indices = np.where((self.annotations['skin_tone'].isin(skin_tone) & \
                           self.annotations['malignant'].isin([di=="malignant" for
di in diagnosis])))[0]
        return Subset(self, indices)

```

```

=====
FILE: DDI_eval_summary.csv
=====
model,preprocessing,ROC_AUC,F1,Balanced_Acc
DeepDerm,images,0.5954422137818773,0.40332640332640335,0.5640381021281726

```

```

=====
FILE: ddi_model.py
=====
"""Code for defining and loading the models we trained."""
import os
import torch
import torchvision

# google drive paths to our models
MODEL_WEB_PATHS = {
# base form of models trained on skin data
'HAM10000': 'https://drive.google.com/uc?id=1ToT8ifJ5lcWh8Ix19ifWlMcMz9UZXcmo',
'DeepDerm': 'https://drive.google.com/uc?id=10Lt11htu9bMPgsE33vZuDiU5Xe4UqKVJ',

# robust training algorithms
'GroupDRO': 'https://drive.google.com/uc?id=193ippDUYpMa0aEyLjd1DNsOiW0aRXL75',
'CORAL':    'https://drive.google.com/uc?id=18rMU0nRd4LiHN9WkXoDROJ2o2sG1_GD8',

```

```
'CDANN': 'https://drive.google.com/uc?id=1PvvgQVqcrth840bFZ3ddLdVSL7NkxiRK',
}
```

```
# thresholds determined by maximizing F1-score on the test split of the train
# dataset for the given algorithm
```

```
MODEL_THRESHOLDS = {
    'HAM10000':0.733,
    'DeepDerm':0.687,
    # robust training algorithms
    'GroupDRO':0.980,
    'CORAL':0.990,
    'CDANN':0.980,
}
```

```
def load_model(model_name, save_dir="DDI-models", download=True, weights_path=None):
    """Load the model and download if necessary. Saves model to provided save
    directory.
```

```
    Args:
```

```
        model_name: One of HAM10000, DeepDerm, GroupDRO, CORAL, CDANN.
        save_dir: Directory where weights are stored or will be downloaded.
        download: If True and weights are missing, download from the web.
        weights_path: Optional path to a local weights file (e.g. from Zenodo).
                     If set, this file is used and save_dir/download are ignored.
```

```
    """
```

```
    if weights_path is not None:
```

```
        model_path = os.path.abspath(weights_path)
        if not os.path.exists(model_path):
            raise FileNotFoundError(f"Weights file not found: {model_path}")
```

```
    else:
```

```
        os.makedirs(save_dir, exist_ok=True)
        model_path = os.path.join(save_dir, f"{model_name.lower()}.pth")
        if not os.path.exists(model_path):
            if not download:
                raise Exception("Model not downloaded and download option not" \
                                " enabled.")
```

```
        else:
```

```
            # Requires installation of gdown (pip install gdown)
```

```
            import gdown
```

```
            gdown.download(MODEL_WEB_PATHS[model_name], model_path)
```

```
    model = torchvision.models.inception_v3(pretrained=False, transform_input=True)
```

```
    model.fc = torch.nn.Linear(2048, 2)
```

```
    model.AuxLogits.fc = torch.nn.Linear(768, 2)
```

```
    state_dict = torch.load(model_path)
```

```
    model.load_state_dict(state_dict)
```

```
    model._ddi_name = model_name
```

```
    model._ddi_threshold = MODEL_THRESHOLDS[model_name]
```

```
    model._ddi_web_path = MODEL_WEB_PATHS[model_name]
```

```
    return model
```

```
=====
FILE: debug_output.txt
=====
```

```
[Binary file]
```

```
=====
FILE: eda_ddi.py
=====
```

```
"""
```

```
Exploratory Data Analysis for the DDI dataset.
```

Analyzes class balance (benign vs malignant), skin tone distribution, and diagnosis – skin tone cross-tabulation. Saves summary stats and plots.

Usage:

```
python eda_ddi.py --data_dir=DDI --out_dir=EDA-output
python eda_ddi.py # uses defaults: data_dir=DDI, out_dir=EDA-output
"""
```

```
import argparse
import os
import sys
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
def load_metadata(data_dir: str, csv_name: str = "ddi_metadata.csv") ->
pd.DataFrame:
```

```
    """Load DDI metadata CSV and normalize malignant column."""
```

```
    csv_path = os.path.join(data_dir, csv_name)
```

```
    if not os.path.exists(csv_path):
```

```
        raise FileNotFoundError(
            f"Metadata not found: {csv_path}\n"
```

```
            "Download DDI from
```

```
https://stanfordaimi.azurewebsites.net/datasets/35866158-8196-48d8-87bf-50dca81df965
"
```

```
)
```

```
df = pd.read_csv(csv_path)
```

```
# Match ddi_dataset.py: ensure 'malignant' column
```

```
if "malignant" not in df.columns and "malignancy(malig=1)" in df.columns:
```

```
    df["malignant"] = (df["malignancy(malig=1)"] == 1).astype(int)
```

```
return df
```

```
def run_eda(df: pd.DataFrame, out_dir: str) -> None:
```

```
    """Compute EDA stats and generate plots."""
```

```

os.makedirs(out_dir, exist_ok=True)
n = len(df)

# --- Class balance (benign vs malignant) ---
class_counts = df["malignant"].value_counts().sort_index()
benign_count = class_counts.get(0, 0)
malignant_count = class_counts.get(1, 0)
benign_pct = 100 * benign_count / n
malignant_pct = 100 * malignant_count / n
imbalance_ratio = max(benign_count, malignant_count) / max(1, min(benign_count,
malignant_count))

class_summary = [
    "=" * 60,
    "CLASS BALANCE (Diagnosis)",
    "=" * 60,
    f" Total samples:      {n}",
    f" Benign (0):          {benign_count:6d} ({benign_pct:5.2f}%)",
    f" Malignant (1):       {malignant_count:6d} ({malignant_pct:5.2f}%)",
    f" Imbalance ratio:     {imbalance_ratio:.2f} (max/min class size)",
    "",
]
lines = "\n".join(class_summary)
print(lines)

# --- Skin tone distribution ---
skin_counts = df["skin_tone"].value_counts().sort_index()
skin_labels = {12: "Fitzpatrick 1-2", 34: "Fitzpatrick 3-4", 56: "Fitzpatrick
5-6"}
skin_summary = [
    "",
    "=" * 60,
    "SKIN TONE DISTRIBUTION",
    "=" * 60,
    f" Total samples:      {n}",
]
for st in [12, 34, 56]:
    c = skin_counts.get(st, 0)
    pct = 100 * c / n
    skin_summary.append(f" {skin_labels[st]:20s} {c:6d} ({pct:5.2f}%)")
skin_summary.append("")
lines_skin = "\n".join(skin_summary)
print(lines_skin)

# --- Cross-tab: diagnosis Ã– skin_tone ---
ct = pd.crosstab(df["malignant"], df["skin_tone"], margins=True)
ct.index = ["Benign", "Malignant", "Total"]
ct.columns = [skin_labels.get(c, str(c)) for c in ct.columns]
cross_tab = [
    "",

```

```

        "=" * 60,
        "CROSS-TAB: Diagnosis ~ Skin tone (counts)",
        "=" * 60,
        ct.to_string(),
        "",
    ]
    lines_cross = "\n".join(cross_tab)
    print(lines_cross)

# Proportions within each skin tone (row per skin tone)
ct_prop = pd.crosstab(df["skin_tone"], df["malignant"], normalize="index") * 100
ct_prop.index = [skin_labels.get(i, i) for i in ct_prop.index]
ct_prop.columns = ["Benign %", "Malignant %"]
prop_tab = [
    "Proportion within each skin tone (%):",
    ct_prop.round(2).to_string(),
    "",
]
print("\n".join(prop_tab))

# --- Save summary to file ---
summary_path = os.path.join(out_dir, "eda_summary.txt")
with open(summary_path, "w") as f:
    f.write("\n".join(class_summary))
    f.write("\n")
    f.write(lines_skin)
    f.write("\n")
    f.write(lines_cross)
    f.write("\n")
    f.write("\n".join(prop_tab))
print(f"Summary saved to {summary_path}")

# --- Plots ---
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# 1. Class balance bar chart
ax = axes[0]
classes = ["Benign", "Malignant"]
counts = [benign_count, malignant_count]
colors = ["#2ecc71", "#e74c3c"]
bars = ax.bar(classes, counts, color=colors, edgecolor="black", linewidth=0.8)
ax.set_ylabel("Count")
ax.set_title("Class balance (diagnosis)")
ax.set_axisbelow(True)
ax.yaxis.grid(True, linestyle="--", alpha=0.7)
for b, c in zip(bars, counts):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height() + max(counts) * 0.02,
            str(c), ha="center", va="bottom", fontsize=11)

# 2. Skin tone distribution

```

```

ax = axes[1]
st_labels = [skin_labels[k] for k in [12, 34, 56]]
st_counts = [skin_counts.get(k, 0) for k in [12, 34, 56]]
colors_st = ["#f1c40f", "#e67e22", "#8e44ad"]
bars = ax.bar(st_labels, st_counts, color=colors_st, edgecolor="black",
linewidth=0.8)
ax.set_ylabel("Count")
ax.set_title("Skin tone distribution")
ax.tick_params(axis="x", rotation=15)
ax.yaxis.grid(True, linestyle="--", alpha=0.7)
for b, c in zip(bars, st_counts):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height() + max(st_counts) *
0.02, str(c), ha="center", va="bottom", fontsize=10)

# 3. Stacked bar: diagnosis by skin tone
ax = axes[2]
st_keys = [12, 34, 56]
benign_by_st = [len(df[(df["skin_tone"] == st) & (df["malignant"] == 0)]) for st
in st_keys]
malignant_by_st = [len(df[(df["skin_tone"] == st) & (df["malignant"] == 1)]) for
st in st_keys]
x = np.arange(len(st_keys))
w = 0.5
ax.bar(x - w / 2, benign_by_st, w, label="Benign", color="#2ecc71",
edgecolor="black", linewidth=0.6)
ax.bar(x - w / 2, malignant_by_st, w, bottom=benign_by_st, label="Malignant",
color="#e74c3c", edgecolor="black", linewidth=0.6)
ax.set_xticks(x)
ax.set_xticklabels([skin_labels[k] for k in st_keys], rotation=15)
ax.set_ylabel("Count")
ax.set_title("Diagnosis by skin tone")
ax.legend(loc="upper right", fontsize=9)
ax.yaxis.grid(True, linestyle="--", alpha=0.7)

plt.tight_layout()
plot_path = os.path.join(out_dir, "eda_plots.png")
plt.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close()
print(f"Plots saved to {plot_path}")

def main():
    parser = argparse.ArgumentParser(description="EDA for DDI dataset (class
balance, skin tone).")
    parser.add_argument("--data_dir", type=str, default="DDI", help="Directory
containing ddi_metadata.csv (and optionally images/).")
    parser.add_argument("--out_dir", type=str, default="EDA-output", help="Directory
to write summary and plots.")
    parser.add_argument("--csv", type=str, default="ddi_metadata.csv",
help="Metadata CSV filename.")

```

```

args = parser.parse_args()

df = load_metadata(args.data_dir, args.csv)
run_eda(df, args.out_dir)

if __name__ == "__main__":
    main()

=====
FILE: eval_all_preprocessed.py
=====
import os
import subprocess
import pickle
import pandas as pd
from concurrent.futures import ProcessPoolExecutor, as_completed
from sklearn.metrics import f1_score, balanced_accuracy_score

# -----
# Configuration
# -----
preprocessed_dirs = [
    "images_bilateral_optimized",
    "images_illumination_comp_optimized",
    "images_illumination_comp+local_contrast_optimized",
    "images_illumination_comp+adaptive_gamma_optimized",
    "images_z_score_norm+adaptive_gamma_optimized",
    "images_z_score_norm+percentile_norm_optimized",
    "images_bilateral+z_score_norm_optimized",
    "images_msrrc+local_contrast_optimized",
    "images_clahe+adaptive_gamma_optimized",
    "images_non_local_means+msrrc_optimized",
    "images_non_local_means+msrrc+adaptive_gamma_optimized",
    "images_non_local_means+illumination_comp+clahe_optimized",
    "images"
]

data_dir = "DDI"
weights_paths = {
    "HAM10000": "DDI-models/ham10000.pth",
    "DeepDerm": "DDI-models/deepderm.pth"
}

use_gpu = True
plot = True # plots will be saved automatically, not shown interactively
summary_csv = "DDI_eval_summary.csv"
max_workers = 3 # Adjust to CPU/GPU resources

```

```

# -----
# Evaluation function
# -----
def run_eval(model_name, weights_path, img_dir):
    eval_dir = f"DDI-results_{img_dir}_{model_name}"
    os.makedirs(eval_dir, exist_ok=True)

    python_exe = r"C:\Users\LeoYa\GitHub\DDI-Code\.venv\Scripts\python.exe"
    cmd = [
        python_exe, "eval_ddi.py",
        "--model", model_name,
        "--weights_path", weights_path,
        "--data_dir", data_dir,
        "--img_dir", img_dir,
        "--eval_dir", eval_dir
    ]
    if use_gpu:
        cmd.append("--use_gpu")
    if plot:
        cmd.append("--plot") # ensure eval_ddi.py saves plot as file

    print(f"Starting: Model={model_name}, Images={img_dir}")
    subprocess.run(cmd, check=True) # raises if eval_ddi.py fails

    eval_file = os.path.join(eval_dir, f"{model_name}-evaluation.pkl")
    if os.path.exists(eval_file):
        with open(eval_file, "rb") as f:
            eval_results = pickle.load(f)

        return {
            "model": model_name,
            "preprocessing": img_dir,
            "ROC_AUC": eval_results.get("ROC_AUC"),
            "F1": f1_score(
                eval_results["true_labels"],
                (eval_results["predicted_labels"] >
eval_results["threshold"]).astype(int)
            ),
            "Balanced_Acc": balanced_accuracy_score(
                eval_results["true_labels"],
                (eval_results["predicted_labels"] >
eval_results["threshold"]).astype(int)
            )
        }
    else:
        print(f"Warning: Evaluation file missing for {model_name} on {img_dir}")
        return None

# -----

```

```

# Main execution (Windows-safe)
# -----
def main():
    results_summary = []

    # create all tasks
    tasks = [(m, w, d) for m, w in weights_paths.items() for d in preprocessed_dirs]

    # run evaluations in parallel
    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        future_to_task = {executor.submit(run_eval, *task): task for task in tasks}

        for future in as_completed(future_to_task):
            task = future_to_task[future]
            try:
                result = future.result()
                if result is not None:
                    results_summary.append(result)
            except Exception as e:
                print(f"Task {task} failed: {e}")

    # save summary CSV
    df = pd.DataFrame(results_summary)
    df.to_csv(summary_csv, index=False)
    print(f"\nSummary CSV saved to {summary_csv}")
    print(df)

# -----
# Entry point
# -----
if __name__ == "__main__":
    main()

```

```

=====
FILE: eval_data.py
=====

```

```

"""

```

Code to load the models trained for our paper,
 "Disparities in Dermatology AI Performance on a Diverse,
 Curated Clinical Image Set".

Examples:

(1) w/command line interface

evaluate DeepDerm on DDI and store results in `DDI-results`

```
>>>python3 eval.py --model=DeepDerm --data_dir=DDI --eval_dir=DDI-results
```

(2) w/python functions

```
>>>import eval
```

```

>>>model = eval.load_model("DeepDerm") # load DeepDerm model
>>>eval_results = eval.eval_model(model, "DDI") # evaluate images in DDI folder

"""
import argparse
import matplotlib.pyplot as plt
import numpy as np
import os
import pickle
from sklearn.metrics import (f1_score, balanced_accuracy_score,
                             classification_report, confusion_matrix, roc_curve, auc)
import torch
import torchvision
from torchvision import transforms, datasets
import tqdm

# google drive paths to models
MODEL_WEB_PATHS = {
    'HAM10000': 'https://drive.google.com/uc?id=1ToT8ifJ5lcWh8Ix19ifWlMcMz9UZXcmo',
    'DeepDerm': 'https://drive.google.com/uc?id=1OLt11htu9bMPgsE33vZuDiU5Xe4UqKVJ',
    # robust training algorithms
    'GroupDRO': 'https://drive.google.com/uc?id=193ippDUYpMaOaEyLjd1DNsOiW0aRXL75',
    'CORAL':    'https://drive.google.com/uc?id=18rMU0nRd4LiHN9WkXoDROJ2o2sG1_GD8',
    'CDANN':    'https://drive.google.com/uc?id=1PvvgQVqcrth840bFZ3ddLdVSL7NkxiRK',
}

# thresholds determined by maximizing F1-score on the test split of the train
# dataset for the given algorithm
MODEL_THRESHOLDS = {
    'HAM10000': 0.733,
    'DeepDerm': 0.687,
    # robust training algorithms
    'GroupDRO': 0.980,
    'CORAL':    0.990,
    'CDANN':    0.980,
}

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model_dir', type=str, default="DDI-models",
                        help="File path for where to save models.")
    parser.add_argument('--model', type=str, default="DeepDerm",
                        help="Name of the model to load (HAM10000, DeepDerm, GroupDRO, CORAL, \"\
                             \" or CDANN).")
    parser.add_argument('--no_download', action='store_true', default=False,
                        help="Set to disable downloading models.")
    parser.add_argument('--data_dir', type=str, default="DDI",
                        help="Folder containing dataset to load. Structure should match the \"\
                             \" root directory in torchvision.datasets.ImageFolder with 2 \"\
                             \" classes: benign (class 0) and malignant (class 1).")

```

```

parser.add_argument('--eval_dir', type=str, default="DDI-results",
                    help="Folder to store evaluation results.")
parser.add_argument('--use_gpu', action='store_true', default=False,
                    help="Set to use GPU for evaluation.")
parser.add_argument('--plot', action='store_true', default=False,
                    help="Set to show ROC plot.")
args = parser.parse_args()
return args

def load_model(model_name, save_dir="DDI-models", download=True):
    """Load the model and download if necessary. Saves model to provided save
    directory."""

    os.makedirs(save_dir, exist_ok=True)
    model_path = os.path.join(save_dir, f"{model_name.lower()}.pth")
    if not os.path.exists(model_path):
        if not download:
            raise Exception("Model not downloaded and download option not" \
                            " enabled.")
        else:
            # Requires installation of gdown (pip install gdown)
            import gdown
            gdown.download(MODEL_WEB_PATHS[model_name], model_path)
    model = torchvision.models.inception_v3(init_weights=False, pretrained=False,
transform_input=True)
    model.fc = torch.nn.Linear(2048, 2)
    model.AuxLogits.fc = torch.nn.Linear(768, 2)
    state_dict = torch.load(model_path)
    model.load_state_dict(state_dict)
    model._ddi_name = model_name
    model._ddi_threshold = MODEL_THRESHOLDS[model_name]
    model._ddi_web_path = MODEL_WEB_PATHS[model_name]
    return model

class ImageFolderWithPaths(datasets.ImageFolder):
    """Custom dataset that includes image file paths. Extends
    torchvision.datasets.ImageFolder
    """

    # override the __getitem__ method. this is the method that dataloader calls
    def __getitem__(self, index):
        # this is what ImageFolder normally returns
        original_tuple = super(ImageFolderWithPaths, self).__getitem__(index)
        # the image file path
        path = os.path.abspath(self.imgs[index][0])
        # make a new tuple that includes original and the path
        tuple_with_path = (original_tuple + (path,))
        return tuple_with_path

```

```

def eval_model(model, image_dir, use_gpu=False, show_plot=False):
    """Evaluate loaded model on provided image dataset. Assumes supplied image
    directory corresponds to `root` input for torchvision.datasets.ImageFolder
    class. Assumes the data is split into binary/malignant labels, as this is
    what our models are trained+evaluated on."""

    use_gpu = (use_gpu and torch.cuda.is_available())
    device = torch.device("cuda") if use_gpu else torch.device("cpu")

    # load dataset
    normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                     std=[0.229, 0.224, 0.225])
    dataset = ImageFolderWithPaths(
        image_dir,
        transforms.Compose([
            transforms.Resize(299),
            transforms.CenterCrop(299),
            transforms.ToTensor(),
            normalize]))
    dataloader = torch.utils.data.DataLoader(
        dataset,
        batch_size=32, shuffle=False,
        num_workers=0, pin_memory=use_gpu)

    # prepare model for evaluation
    model.to(device).eval()

    # log output for all images in dataset
    hat, star, all_paths = [], [], []
    for batch in tqdm.tqdm(enumerate(dataloader)):
        i, (images, target, paths) = batch
        images = images.to(device)
        target = target.to(device)

        with torch.no_grad():
            output = model(images)

        hat.append(output[:,1].detach().cpu().numpy())
        star.append(target.cpu().numpy())
        all_paths.append(paths)

    hat = np.concatenate(hat)
    star = np.concatenate(star)
    all_paths = np.concatenate(all_paths)
    threshold = model._ddi_threshold
    m_name = model._ddi_name
    m_web_path = model._ddi_web_path

    report = classification_report(star, (hat>threshold).astype(int),
        target_names=["benign", "malignant"])

```

```

fpr, tpr, _ = roc_curve(star, hat, pos_label=1,
                        sample_weight=None,
                        drop_intermediate=True)
auc_est = auc(fpr, tpr)

if show_plot:
    _=plt.plot(fpr, tpr,
               color="blue", linestyle="-", linewidth=2,
               marker="o", markersize=2,
               label=f"AUC={auc_est:.3f}")[0]
    plt.show()
    plt.close()

eval_results = {'predicted_labels':hat, # predicted labels by model
                'true_labels':star,    # true labels
                'images':all_paths,    # image paths
                'report':report,       # sklearn classification report
                'ROC_AUC':auc_est,     # ROC-AUC
                'threshold':threshold, # >= threshold ==> malignant
                'model':m_name,        # model name
                'web_path':m_web_path, # web link to download model
                }

return eval_results

```

```

if __name__ == '__main__':
    # get arguments from command line
    args = get_args()
    # load model and download if necessary
    model = load_model(args.model,
                       save_dir=args.model_dir, download=not args.no_download)
    # evaluate results on data
    eval_results = eval_model(model, args.data_dir, use_gpu=args.use_gpu,
                              show_plot=args.plot)

    # save evaluation results in a .pkl file
    if args.eval_dir:
        os.makedirs(args.eval_dir, exist_ok=True)
        eval_save_path = os.path.join(args.eval_dir,
                                       f"{args.model}-evaluation.pkl")
        with open(eval_save_path, 'wb') as f:
            pickle.dump(eval_results, f)

    # load results with:
    #with open(eval_save_path, 'rb') as f:
    #    results = pickle.load(f)

```

=====

FILE: eval_ddi.py

=====

"""

Code to evaluate the models trained for our paper,
"Disparities in Dermatology AI Performance on a Diverse,
Curated Clinical Image Set",
on the DDI dataset.

Note: assumes DDI data is organized as

./DDI

 /images

 /000001.png

 /000002.png

 ...

 /ddi_metadata.csv

(After downloading from the Stanford AIMI repository, this requires moving all .png
files into a new subdirectory titled "images".)

Examples:

(1) w/command line interface

evaluate DeepDerm on DDI and store results in `DDI-results`

>>>python3 eval_ddi.py --model=DeepDerm --data_dir=DDI --eval_dir=DDI-results

(2) w/python functions

>>>import eval_ddi

>>>import ddi_model

>>>model = ddi_model.load_model("DeepDerm") # load DeepDerm model

>>>eval_results = eval_ddi.eval_model(model, "DDI") # evaluate images in DDI folder

(3) using local weights (e.g. from Zenodo)

>>>python3 eval_ddi.py --model=DeepDerm --weights_path=path/to/deepderm.pth

--data_dir=DDI

"""

import argparse

from ddi_dataset import DDI_Dataset

from ddi_model import load_model

import matplotlib.pyplot as plt

import numpy as np

import os

import pickle

from sklearn.metrics import (f1_score, balanced_accuracy_score,
 classification_report, confusion_matrix, roc_curve, auc)

import torch

```

import tqdm

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model_dir', type=str, default="DDI-models",
        help="File path for where to save models.")
    parser.add_argument('--model', type=str, default="DeepDerm",
        help="Name of the model to load (HAM10000, DeepDerm, GroupDRO, CORAL, \"\
        \" or CDANN).")
    parser.add_argument('--no_download', action='store_true', default=False,
        help="Set to disable downloading models.")
    parser.add_argument('--weights_path', type=str, default=None,
        help="Path to local weights file (e.g. from Zenodo). If set, model is loaded
from this file instead of model_dir.")
    parser.add_argument('--data_dir', type=str, default="DDI",
        help="Folder containing dataset to load. Structure should be: (1)
`[data_dir]/images` contains all images; (2) `[data_dir]/ddi_metadata.csv` contains
the CSV metadata for the DDI dataset")
    parser.add_argument('--img_dir', type=str, default="images",
        help="Name of the subdirectory in data_dir containing images (default:
'images').")
    parser.add_argument('--eval_dir', type=str, default="DDI-results",
        help="Folder to store evaluation results.")
    parser.add_argument('--use_gpu', action='store_true', default=False,
        help="Set to use GPU for evaluation.")
    parser.add_argument('--plot', action='store_true', default=False,
        help="Set to show ROC plot.")
    parser.add_argument('--shap', action='store_true', default=False,
        help="Run SHAP explanations on a subset of images.")
    parser.add_argument('--shap_samples', type=int, default=20,
        help="Number of images to explain with SHAP (default=20).")
    args = parser.parse_args()
    return args

def eval_model(model, dataset, use_gpu=False, show_plot=False, num_workers=0):
    """Evaluate loaded model on provided image dataset. Assumes supplied image
    directory corresponds to `root` input for torchvision.datasets.ImageFolder
    class. Assumes the data is split into binary/malignant labels, as this is
    what our models are trained+evaluated on."""

    use_gpu = (use_gpu and torch.cuda.is_available())
    device = torch.device("cuda") if use_gpu else torch.device("cpu")

    # load dataset
    dataloader = torch.utils.data.DataLoader(
        dataset,
        batch_size=32, shuffle=False,
        num_workers=num_workers, pin_memory=use_gpu)

```

```

# prepare model for evaluation
model.to(device).eval()

# log output for all images in dataset
hat, star, all_paths = [], [], []
for batch in tqdm.tqdm(enumerate(dataloader)):
    i, (paths, images, target, skin_tone) = batch
    images = images.to(device)
    target = target.to(device)

    with torch.no_grad():
        output = model(images)

    hat.append(output[:,1].detach().cpu().numpy())
    star.append(target.cpu().numpy())
    all_paths.append(paths)

hat = np.concatenate(hat)
star = np.concatenate(star)
all_paths = np.concatenate(all_paths)
threshold = model._ddi_threshold
m_name = model._ddi_name
m_web_path = model._ddi_web_path

report = classification_report(star, (hat>threshold).astype(int),
    target_names=["benign","malignant"])
fpr, tpr, _ = roc_curve(star, hat, pos_label=1,
    sample_weight=None,
    drop_intermediate=True)

auc_est = auc(fpr, tpr)

if show_plot:
    _=plt.plot(fpr, tpr,
        color="blue", linestyle="-", linewidth=2,
        marker="o", markersize=2,
        label=f"AUC={auc_est:.3f}")
    if args.eval_dir:
        os.makedirs(args.eval_dir, exist_ok=True)
        plot_path = os.path.join(args.eval_dir,
f"{m_name}_ROC_{os.path.basename(dataset.root)}.png")
        plt.savefig(plot_path)
        plt.close()

eval_results = {'predicted_labels':hat, # predicted labels by model
    'true_labels':star, # true labels
    'images':all_paths, # image paths
    'report':report, # sklearn classification report
    'ROC_AUC':auc_est, # ROC-AUC
    'threshold':threshold, # >= threshold ==> malignant
    'model':m_name, # model name

```

```

        'web_path':m_web_path, # web link to download model
    }

    return eval_results

if __name__ == '__main__':
    # get arguments from command line
    args = get_args()
    # load model and download if necessary (or from --weights_path if provided)
    model = load_model(args.model,
        save_dir=args.model_dir, download=not args.no_download,
        weights_path=args.weights_path)
    from ddi_dataset import test_transform
    # load DDI dataset
    dataset = DDI_Dataset(args.data_dir, img_dirname=args.img_dir,
transform=test_transform)
    # evaluate results on data
    eval_results = eval_model(model, dataset,
        use_gpu=args.use_gpu, show_plot=args.plot)
    # save evaluation results in a .pkl file
    if args.eval_dir:
        os.makedirs(args.eval_dir, exist_ok=True)
        eval_save_path = os.path.join(args.eval_dir,
            f"{args.model}-evaluation.pkl")
        with open(eval_save_path, 'wb') as f:
            pickle.dump(eval_results, f)

        # load results with:
        #with open(eval_save_path, 'rb') as f:
        #    results = pickle.load(f)

```

=====

FILE: evaluate_fairness.py

=====

```

import argparse
import pickle
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import (
    roc_auc_score, roc_curve, precision_recall_curve,
    confusion_matrix, classification_report,

```

```

    accuracy_score, precision_score, recall_score, f1_score
)
from scipy.special import expit

# -----
# Utility Functions
# -----

def normalize_filename(x):
    return os.path.splitext(os.path.basename(str(x)))[0]

def bootstrap_auc_ci(y_true, y_score, n_bootstrap=1000, seed=42):
    """
    Bootstrapped AUC with guaranteed n_bootstrap samples, resampling within the
    input arrays.
    Skips samples where all labels are the same.
    """
    rng = np.random.default_rng(seed)
    y_true = np.array(y_true)
    y_score = np.array(y_score)
    aucs = []
    attempts = 0
    max_attempts = n_bootstrap * 10

    while len(aucs) < n_bootstrap and attempts < max_attempts:
        idx = rng.integers(0, len(y_true), len(y_true))
        if len(np.unique(y_true[idx])) < 2:
            attempts += 1
            continue
        aucs.append(roc_auc_score(y_true[idx], y_score[idx]))

    if len(aucs) == 0:
        raise ValueError("Unable to compute any bootstrap AUCs. Check subgroup size
or labels.")

    mean_auc = np.mean(aucs)
    lower = np.percentile(aucs, 2.5)
    upper = np.percentile(aucs, 97.5)
    return mean_auc, lower, upper, np.array(aucs)

def bootstrap_auc_gap(y_true, y_score, groups, group1, group2, n_bootstrap=1000,
seed=42):
    """
    Directly bootstrap the AUC gap (group1 - group2) per iteration.
    Ensures CI reflects variability in both groups simultaneously.
    """
    rng = np.random.default_rng(seed)
    y_true = np.array(y_true)
    y_score = np.array(y_score)
    groups = np.array(groups)

```

```

mask1 = groups == group1
mask2 = groups == group2
y1, s1 = y_true[mask1], y_score[mask1]
y2, s2 = y_true[mask2], y_score[mask2]

gap_dist = []
attempts = 0
max_attempts = n_bootstrap * 10

while len(gap_dist) < n_bootstrap and attempts < max_attempts:
    idx1 = rng.integers(0, len(y1), len(y1))
    idx2 = rng.integers(0, len(y2), len(y2))
    if len(np.unique(y1[idx1])) < 2 or len(np.unique(y2[idx2])) < 2:
        attempts += 1
        continue
    auc1 = roc_auc_score(y1[idx1], s1[idx1])
    auc2 = roc_auc_score(y2[idx2], s2[idx2])
    gap_dist.append(auc1 - auc2)

if len(gap_dist) == 0:
    raise ValueError("Unable to compute any bootstrap AUC gaps. Check subgroup
sizes.")

gap_dist = np.array(gap_dist)
return np.mean(gap_dist), np.percentile(gap_dist, 2.5), np.percentile(gap_dist,
97.5), gap_dist

def compute_tpr_gap(y_true, y_pred, groups):
    tprs = {}
    for g in np.unique(groups):
        mask = groups == g
        cm = confusion_matrix(y_true[mask], y_pred[mask], labels=[0,1])
        tn, fp, fn, tp = cm.ravel()
        tpr = tp / (tp + fn) if (tp + fn) > 0 else 0
        tprs[g] = tpr
    gap = max(tprs.values()) - min(tprs.values())
    return tprs, gap

def gap_significance_test(baseline_gap_dist, new_gap_dist):
    """
    Two-sided p-value: fraction of bootstrap differences less than 0 or greater than
0
    """
    diff = np.array(baseline_gap_dist) - np.array(new_gap_dist)
    p_value = 2 * min(np.mean(diff <= 0), np.mean(diff >= 0))
    return p_value

def skin_group_label(g):
    mapping = {12: "1-2", 34: "3-4", 56: "5-6"}

```

```

        return mapping.get(g, str(g))

# -----
# Main Evaluation
# -----

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--pkl", required=True)
    parser.add_argument("--metadata", required=True)
    parser.add_argument("--out", required=True)
    parser.add_argument("--compare", type=str, default=None)
    parser.add_argument("--baseline_gap_file", type=str, default=None)
    args = parser.parse_args()

    os.makedirs(args.out, exist_ok=True)

    # -----
    # Load pickle data
    # -----
    with open(args.pkl, "rb") as f:
        data = pickle.load(f)
    if not isinstance(data, dict):
        raise ValueError("Expected dictionary-format pickle.")

    df = pd.DataFrame({
        "y_true": data["true_labels"],
        "y_score": data["predicted_labels"],
        "DDI_file": data["images"]
    })
    df["DDI_file"] = df["DDI_file"].apply(normalize_filename)
    df["y_prob"] = expit(df["y_score"])

    # -----
    # Load metadata and merge
    # -----
    metadata = pd.read_csv(args.metadata)
    metadata["DDI_file"] = metadata["DDI_file"].apply(normalize_filename)
    df = df.merge(metadata[["DDI_file", "skin_tone"]], on="DDI_file", how="left")

    missing = df["skin_tone"].isnull().sum()
    if missing > 0:
        raise ValueError(f"Missing skin tone labels after merge: {missing}")

    y_true = df["y_true"].values
    y_prob = df["y_prob"].values
    groups = df["skin_tone"].values

    # -----
    # Optimal threshold (Youden J)

```

```

# -----
fpr_all, tpr_all, thresholds = roc_curve(y_true, y_prob)
best_idx = np.argmax(tpr_all - fpr_all)
optimal_thresh = thresholds[best_idx]
df["y_pred"] = (df["y_prob"] >= optimal_thresh).astype(int)
y_pred = df["y_pred"].values

print(f"\nOptimal threshold (Youden J): {optimal_thresh:.4f}")
best_sensitivity = tpr_all[best_idx]
best_specificity = 1 - fpr_all[best_idx]
print(f"Sensitivity at Youden threshold: {best_sensitivity:.4f}")
print(f"Specificity at Youden threshold: {best_specificity:.4f}")

# -----
# Overall performance
# -----
print("\n--- Overall Performance ---")
print(classification_report(y_true, y_pred))
overall_auc = roc_auc_score(y_true, y_prob)
print(f"Overall ROC AUC (raw, non-bootstrapped): {overall_auc:.4f}")

# -----
# Per-group metrics + bootstrap
# -----
results = []
auc_distributions = {}
raw_aucs = {}
roc_data = {}
pr_data = {}

for g in sorted(np.unique(groups)):
    mask = groups == g
    label = skin_group_label(g)

    # Raw AUC
    raw_auc = roc_auc_score(y_true[mask], y_prob[mask])
    raw_aucs[label] = raw_auc
    print(f"Raw AUC Fitzpatrick {label}: {raw_auc:.4f}")

    # Bootstrap AUC
    mean_auc, ci_lower, ci_upper, auc_dist = bootstrap_auc_ci(y_true[mask],
y_prob[mask], n_bootstrap=1000)
    auc_distributions[label] = auc_dist

    acc = accuracy_score(y_true[mask], y_pred[mask])
    prec = precision_score(y_true[mask], y_pred[mask], zero_division=0)
    rec = recall_score(y_true[mask], y_pred[mask], zero_division=0)
    f1 = f1_score(y_true[mask], y_pred[mask], zero_division=0)

    print(f"\n--- Fitzpatrick {label} ---")

```

```

print(f"AUC: {mean_auc:.4f} (95% CI: {ci_lower:.3f}-{ci_upper:.3f})")
print(f"Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1:
{f1:.4f}")

```

```

results.append({
    "skin_group": label,
    "raw_auc": raw_auc,
    "auc": mean_auc,
    "auc_ci_lower": ci_lower,
    "auc_ci_upper": ci_upper,
    "accuracy": acc,
    "precision": prec,
    "recall": rec,
    "f1": f1
})

```

```

# ROC per group

```

```

fpr_g, tpr_g, _ = roc_curve(y_true[mask], y_prob[mask])
roc_data[label] = (fpr_g, tpr_g, mean_auc)
plt.figure()
plt.plot(fpr_g, tpr_g, label=f"AUC={mean_auc:.3f}")
plt.title(f"ROC Curve - Fitzpatrick {label}")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.savefig(os.path.join(args.out, f"roc_skin_{label}.png"))
plt.close()

```

```

# PR per group

```

```

precision_g, recall_g, _ = precision_recall_curve(y_true[mask],
y_prob[mask])
pr_data[label] = (recall_g, precision_g)
plt.figure()
plt.plot(recall_g, precision_g)
plt.title(f"Precision-Recall - Fitzpatrick {label}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.savefig(os.path.join(args.out, f"pr_skin_{label}.png"))
plt.close()

```

```

# Confusion matrix per group

```

```

cm = confusion_matrix(y_true[mask], y_pred[mask], labels=[0,1])
plt.figure()
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=["Predicted Benign", "Predicted Malignant"],
            yticklabels=["Actual Benign", "Actual Malignant"],
            cmap="Blues")
plt.title(f"Confusion Matrix - Fitzpatrick {label}")
plt.savefig(os.path.join(args.out, f"confusion_matrix_skin_{label}.png"))
plt.close()

```

```

results_df = pd.DataFrame(results)
results_df.to_csv(os.path.join(args.out, "fairness_metrics.csv"), index=False)

# -----
# Bootstrapped AUC Gap (1-2 minus 5-6)
# -----
if 12 in groups and 56 in groups:
    gap_mean, gap_lower, gap_upper, gap_dist = bootstrap_auc_gap(
        y_true, y_prob, groups, 12, 56, n_bootstrap=1000
    )
    print(f"\nBootstrapped AUC Gap (1-2 minus 5-6): {gap_mean:.4f} (95% CI:
{gap_lower:.3f}-{gap_upper:.3f})")
    np.save(os.path.join(args.out, "gap_distribution_1-2_vs_5-6.npy"), gap_dist)
# -----
# AUC Ratio Plot: Fitzpatrick 5-6 vs 1-2
# -----
if "1-2" in auc_distributions and "5-6" in auc_distributions:
    auc_12 = auc_distributions["1-2"]
    auc_56 = auc_distributions["5-6"]

    # Compute ratio per bootstrap iteration
    ratio_56_to_12 = auc_56 / auc_12

    mean_ratio = np.mean(ratio_56_to_12)
    ci_lower_ratio = np.percentile(ratio_56_to_12, 2.5)
    ci_upper_ratio = np.percentile(ratio_56_to_12, 97.5)

    print(f"\nMean AUC ratio (5-6 / 1-2): {mean_ratio:.3f} (95% CI:
{ci_lower_ratio:.3f}-{ci_upper_ratio:.3f})")

    # Plot histogram
    plt.figure(figsize=(7,5))
    plt.hist(ratio_56_to_12, bins=30, color="lightcoral", edgecolor="black")
    plt.axvline(mean_ratio, color="blue", linestyle="--",
label=f"Mean={mean_ratio:.3f}")
    plt.title("Bootstrap AUC Ratio: Fitzpatrick 5-6 / 1-2")
    plt.xlabel("AUC Ratio")
    plt.ylabel("Frequency")
    plt.legend()
    plt.savefig(os.path.join(args.out, "auc_ratio_5-6_vs_1-2.png"))
    plt.close()

# -----
# Percent Decrease in Disparity vs Baseline
# -----
if args.baseline_gap_file:
    baseline_gap_dist = np.load(args.baseline_gap_file)

    # --- Compute means ---

```

```

baseline_mean = np.mean(baseline_gap_dist)
new_mean = np.mean(gap_dist)

# --- Percent decrease from means ---
percent_decrease = ((baseline_mean - new_mean) / baseline_mean) * 100

print(f"\nBaseline mean gap: {baseline_mean:.4f}")
print(f"New mean gap: {new_mean:.4f}")
print(f"Percent decrease in disparity: {percent_decrease:.2f}%")

# --- Bootstrap CI with FIXED denominator ---
reduction_dist = (
    (baseline_gap_dist - gap_dist) / baseline_mean
) * 100

ci_lower_decrease = np.percentile(reduction_dist, 2.5)
ci_upper_decrease = np.percentile(reduction_dist, 97.5)

print(f"95% CI for percent decrease: "
      f"{ci_lower_decrease:.2f}% to {ci_upper_decrease:.2f}%")

# --- Plot distribution ---
plt.figure(figsize=(7,5))
plt.hist(reduction_dist, bins=30, color="lightgreen", edgecolor="black")
plt.axvline(percent_decrease, color="blue", linestyle="--",
            label=f"Mean={percent_decrease:.2f}%")
plt.title("Percent Decrease in AUC Gap (Preprocessed vs Baseline)")
plt.xlabel("Percent Decrease")
plt.ylabel("Frequency")
plt.legend()
plt.savefig(os.path.join(args.out, "percent_decrease_gap.png"))
plt.close()
if args.baseline_gap_file:
    baseline_gap_dist = np.load(args.baseline_gap_file)
    p_val = gap_significance_test(baseline_gap_dist, gap_dist)
    print(f"Gap reduction p-value vs baseline: {p_val:.4f}")
    if p_val < 0.05:
        print("Gap reduction is statistically significant (p < 0.05)")
    else:
        print("Gap reduction is NOT statistically significant (p >= 0.05)")

# -----
# Combined ROC and PR plots
# -----
plt.figure()
plt.plot(fpr_all, tpr_all, label=f"Overall (AUC={overall_auc:.3f})",
linewidth=2)
for label, (fpr_g, tpr_g, mean_auc) in roc_data.items():
    plt.plot(fpr_g, tpr_g, label=f"{label} (AUC={mean_auc:.3f})")
plt.title("ROC Curves by Fitzpatrick Group")

```

```

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.savefig(os.path.join(args.out, "roc_combined.png"))
plt.close()

plt.figure()
precision_all, recall_all, _ = precision_recall_curve(y_true, y_prob)
plt.plot(recall_all, precision_all, label="Overall", linewidth=2)
for label, (recall_g, precision_g) in pr_data.items():
    plt.plot(recall_g, precision_g, label=label)
plt.title("Precision-Recall by Fitzpatrick Group")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend()
plt.savefig(os.path.join(args.out, "pr_combined.png"))
plt.close()

# Bootstrap AUC bar plot
plt.figure(figsize=(7,5))
means = results_df["auc"]
lowers = results_df["auc_ci_lower"]
uppers = results_df["auc_ci_upper"]
errors = [means - lowers, uppers - means]
plt.bar(results_df["skin_group"], means, yerr=errors, capsize=6,
color="skyblue", edgecolor="black")
plt.ylabel("ROC AUC")
plt.title("Per-Group ROC AUC with 95% CI")
plt.ylim(0,1.05)
plt.savefig(os.path.join(args.out, "auc_bar_with_ci.png"))
plt.close()

# -----
# Compare with another model if provided
# -----
if args.compare:
    other = pd.read_csv(args.compare)
    merged = results_df.merge(other, on="skin_group",
suffixes=("_Model1", "_Model2"))
    print("\nComparison with second model:")

print(merged[["skin_group", "auc_Model1", "auc_Model2", "recall_Model1", "recall_Model2"
]])
    merged.to_csv(os.path.join(args.out, "model_comparison.csv"), index=False)

    print("\nEvaluation complete. All metrics and plots saved.")

if __name__ == "__main__":
    main()

```

```

=====
FILE: generate_preprocessed.py
=====
import cv2
import numpy as np
import os
import glob
from multiprocessing import Pool, cpu_count
import tqdm
import argparse

def apply_clahe(img, clip_limit=2.0):
    """Apply CLAHE to L channel of LAB image."""
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=(8, 8))
    l = clahe.apply(l)
    lab = cv2.merge((l, a, b))
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

def apply_adaptive_gamma(img):
    """Apply Adaptive Gamma Correction."""

    # Convert to HSV to get Value channel (brightness)
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)

    # Compute mean brightness
    m = np.mean(v) / 255.0

    # Avoid division by zero or log of zero
    m = np.clip(m, 1e-5, 1.0)

    # Calculate gamma: mapping mean brightness to 0.5
    # log(0.5) / log(m)
    # If m is 0.5, gamma is 1.
    # If m < 0.5 (dark), gamma < 1 (brighten).
    # If m > 0.5 (bright), gamma > 1 (darken).
    gamma = np.log(0.5) / np.log(m)

    # Apply gamma correction to V channel
    # Standard: I_out = I_in ^ gamma
    table = np.array(
        [((i / 255.0) ** gamma) * 255 for i in np.arange(0, 256)]
    ).astype("uint8")

    v_corrected = cv2.LUT(v, table)

    hsv_corrected = cv2.merge((h, s, v_corrected))

```

```

    return cv2.cvtColor(hsv_corrected, cv2.COLOR_HSV2BGR)

def apply_white_balance(img):
    """Apply Gray World White Balance (LAB-based)."""

    result = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

    avg_a = np.average(result[:, :, 1])
    avg_b = np.average(result[:, :, 2])

    result[:, :, 1] = result[:, :, 1] - (
        (avg_a - 128) * (result[:, :, 0] / 255.0) * 1.1
    )
    result[:, :, 2] = result[:, :, 2] - (
        (avg_b - 128) * (result[:, :, 0] / 255.0) * 1.1
    )

    result = cv2.cvtColor(result, cv2.COLOR_LAB2BGR)

    return result

def apply_white_balance_simple(img):
    """Simple Gray World: Scale channels so means are equal."""

    b, g, r = cv2.split(img)

    m_b = np.mean(b)
    m_g = np.mean(g)
    m_r = np.mean(r)

    avg = (m_b + m_g + m_r) / 3

    # Scale factors
    k_b = avg / m_b if m_b > 0 else 1
    k_g = avg / m_g if m_g > 0 else 1
    k_r = avg / m_r if m_r > 0 else 1

    b = cv2.convertScaleAbs(b, alpha=k_b)
    g = cv2.convertScaleAbs(g, alpha=k_g)
    r = cv2.convertScaleAbs(r, alpha=k_r)

    return cv2.merge((b, g, r))

def apply_clahe_adaptive_gamma(img):
    """Apply CLAHE -> Adaptive Gamma."""
    img = apply_clahe(img)
    img = apply_adaptive_gamma(img)

```

```

    return img

# -----
# New preprocessing methods
# -----

def apply_msrrcr(
    img: np.ndarray,
    gamma=128,
    scales=(15, 80, 250),
    alpha=125.0,
    beta=46.0,
    gain=1.0,
    offset=0.0
) -> np.ndarray:
    """
    MSRCR (Multi-Scale Retinex with Color Restoration).
    - Retinex:  $\sum_i (\log(I) - \log(\text{blur}(I, \text{scale}_i)))$ 
    - Color restoration:  $\beta * (\log(\alpha * I) - \log(\sum_{\text{channels}}(I)))$ 
    Notes:
    - This is a common practical MSRCR variant. Parameters can be tuned.
    """
    gain = gamma / 128.0
    img_f = img.astype(np.float32) + 1.0 # avoid log(0)
    retinex = np.zeros_like(img_f)

    for s in scales:
        blur = cv2.GaussianBlur(img_f, (0, 0), sigmaX=float(s), sigmaY=float(s))
        retinex += np.log(img_f) - np.log(blur + 1.0)

    retinex /= float(len(scales))

    # Color restoration
    sum_channels = np.sum(img_f, axis=2, keepdims=True)
    color_restoration = beta * (np.log(alpha * img_f) - np.log(sum_channels + 1.0))

    msrrcr = gain * (retinex * color_restoration) + offset

    # Per-channel normalization to 0..255
    out = np.zeros_like(msrrcr)
    for c in range(3):
        ch = msrrcr[:, :, c]
        ch = ch - np.min(ch)
        denom = np.max(ch) + 1e-6
        ch = (ch / denom) * 255.0
        out[:, :, c] = ch

    return np.clip(out, 0, 255).astype(np.uint8)

```

```

def apply_homomorphic(
    img: np.ndarray,
    cutoff=0.35,
    order=2.0,
    boost=1.6
) -> np.ndarray:
    """
    Homomorphic filtering for illumination normalization (on V channel).
    Steps:
    - work in log domain
    - FFT
    - apply high-pass filter (Butterworth-like radial)
    - inverse FFT
    - exp + normalize
    """
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV).astype(np.float32)
    h, s, v = cv2.split(hsv)

    v = np.clip(v, 1.0, 255.0)
    v_log = np.log(v)

    # FFT
    v_fft = np.fft.fft2(v_log)
    v_fft_shift = np.fft.fftshift(v_fft)

    rows, cols = v.shape
    cy, cx = rows // 2, cols // 2
    y = np.arange(rows) - cy
    x = np.arange(cols) - cx
    X, Y = np.meshgrid(x, y)
    D = np.sqrt(X * X + Y * Y)

    D0 = cutoff * np.max(D) # cutoff as fraction of max radius
    D0 = max(D0, 1e-6)

    # High-pass Butterworth-like
    H = 1.0 - 1.0 / (1.0 + (D / D0) ** (2.0 * order))

    # Apply filter with boost
    v_filt = v_fft_shift * (1.0 + (boost - 1.0) * H)

    # Inverse FFT
    v_ishift = np.fft.ifftshift(v_filt)
    v_ifft = np.fft.ifft2(v_ishift)
    v_out = np.real(v_ifft)

    # Back from log domain
    v_out = np.exp(v_out)

    # Normalize to 0..255

```

```

v_out = v_out - np.min(v_out)
v_out = v_out / (np.max(v_out) + 1e-6)
v_out = (v_out * 255.0).astype(np.uint8)

hsv_out = cv2.merge((h.astype(np.uint8), s.astype(np.uint8), v_out))
return cv2.cvtColor(hsv_out, cv2.COLOR_HSV2BGR)

```

```

def apply_percentile_norm(
    img: np.ndarray,
    range_=98.0
) -> np.ndarray:
    """
    Robust brightness normalization using percentiles (on V channel).
    Clips and stretches brightness so [low_p, high_p] maps to [0,255].
    """
    low_p = (100.0 - range_) / 2.0
    high_p = 100.0 - low_p
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)

    v_f = v.astype(np.float32)
    lo = np.percentile(v_f, low_p)
    hi = np.percentile(v_f, high_p)

    if hi <= lo + 1e-6:
        return img

    v_f = (v_f - lo) / (hi - lo)
    v_f = np.clip(v_f, 0.0, 1.0) * 255.0
    v_out = v_f.astype(np.uint8)

    hsv_out = cv2.merge((h, s, v_out))
    return cv2.cvtColor(hsv_out, cv2.COLOR_HSV2BGR)

def apply_local_contrast(
    img: np.ndarray,
    sigma=2.0,
    strength=1.25
) -> np.ndarray:
    """
    Local feature enhancement via unsharp masking on luminance (L in LAB).
    - strength > 1 boosts local contrast
    """
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB).astype(np.float32)
    l, a, b = cv2.split(lab)

    blur = cv2.GaussianBlur(l, (0, 0), sigmaX=float(sigma), sigmaY=float(sigma))
    detail = l - blur

```

```

l_enh = 1 + strength * detail

l_enh = np.clip(l_enh, 0, 255)
lab_out = cv2.merge((l_enh, a, b)).astype(np.uint8)
return cv2.cvtColor(lab_out, cv2.COLOR_LAB2BGR)

def apply_erythema_enhance(
    img: np.ndarray,
    strength=0.6,
    softness=6.0
) -> np.ndarray:
    """
    Medical-ish erythema emphasis.
    Computes an erythema proxy index and boosts redness in high-index areas.

    Proxy index (simple, fast):
         $EI = 2R - G - B$ 
    Then:
        mask = sigmoid(normalized_EI)
        R += strength * mask * (255 - R)
        G,B slightly decreased in those regions

    This is not a diagnostic tool. It's a visualization-style enhancement.
    """
    bgr = img.astype(np.float32)
    b, g, r = cv2.split(bgr)

    ei = 2.0 * r - g - b # erythema proxy
    # Normalize EI to 0..1 robustly
    ei_lo = np.percentile(ei, 5.0)
    ei_hi = np.percentile(ei, 95.0)
    if ei_hi <= ei_lo + 1e-6:
        return img

    ei_n = (ei - ei_lo) / (ei_hi - ei_lo)
    ei_n = np.clip(ei_n, 0.0, 1.0)

    # Sigmoid mask for smooth emphasis
    mask = 1.0 / (1.0 + np.exp(-softness * (ei_n - 0.5)))

    # Apply adjustments
    r_out = r + strength * mask * (255.0 - r)
    g_out = g * (1.0 - 0.15 * strength * mask)
    b_out = b * (1.0 - 0.15 * strength * mask)

    out = cv2.merge((b_out, g_out, r_out))
    return np.clip(out, 0, 255).astype(np.uint8)

def apply_illumination_comp(

```

```

    img: np.ndarray,
    sigma=50.0,
    eps=1e-6
) -> np.ndarray:
    """
    Illumination compensation via large-scale Gaussian illumination estimation.
    Steps:
    - Convert to LAB
    - Estimate illumination via heavy Gaussian blur on L channel
    - Divide original L by illumination
    - Normalize back to 0..255
    """
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB).astype(np.float32)
    l, a, b = cv2.split(lab)
    # Estimate illumination field
    illum = cv2.GaussianBlur(l, (0, 0), sigmaX=float(sigma), sigmaY=float(sigma))
    # Avoid divide-by-zero
    illum = illum + eps
    # Remove illumination
    l_corr = l / illum
    l_corr = l_corr - np.min(l_corr)
    l_corr = l_corr / (np.max(l_corr) + eps)
    l_corr = l_corr * 255.0
    lab_out = cv2.merge((l_corr, a, b)).astype(np.uint8)
    return cv2.cvtColor(lab_out, cv2.COLOR_LAB2BGR)

def apply_z_score_norm(
    img: np.ndarray,
    scale=1.0,
    eps=1e-6
) -> np.ndarray:
    """
    Per-channel Z-score normalization.
    - Subtract mean
    - Divide by std (adjusted by scale)
    - Rescale to 0..255 for image saving
    """

    img_f = img.astype(np.float32)

    out = np.zeros_like(img_f)

    for c in range(3):
        channel = img_f[:, :, c]
        mean = np.mean(channel)
        std = np.std(channel)

        std = max(std * scale, eps)

        z = (channel - mean) / std

```

```

        # Normalize to 0..255 for saving
        z = z - np.min(z)
        z = z / (np.max(z) + eps)
        z = z * 255.0

        out[:, :, c] = z

    return np.clip(out, 0, 255).astype(np.uint8)

# -----
# Additional New Methods
# -----

def apply_color_constancy(img, power=6, sigma=0):
    """
    Shades of Gray color constancy algorithm.
    Generalization of gray world and white patch.

    power=1 -> gray world
    power=inf -> white patch

    Optionally applies Gaussian smoothing before computing
    the Minkowski norm.
    """
    img_float = img.astype(np.float32) + 1e-6

    # Optional smoothing
    if sigma > 0:
        img_float = cv2.GaussianBlur(img_float, (0, 0), sigma)

    # Minkowski norm across spatial dimensions
    norm = np.power(np.mean(np.power(img_float, power), axis=(0, 1)), 1/power)

    # Normalize each channel
    result = img.astype(np.float32)
    for i in range(3):
        if norm[i] > 0:
            result[:, :, i] = result[:, :, i] / norm[i] * np.mean(norm)

    return np.clip(result, 0, 255).astype(np.uint8)

def apply_lab_color_normalization(img, target_a=128, target_b=128):
    """
    Normalize a and b channels in LAB color space.

    Shifts the mean of the chromatic channels (a and b)
    to a specified target value (default centers).
    """

```

```

lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB).astype(np.float32)

# Shift a and b channels to target mean
lab[:, :, 1] = lab[:, :, 1] - np.mean(lab[:, :, 1]) + target_a
lab[:, :, 2] = lab[:, :, 2] - np.mean(lab[:, :, 2]) + target_b

lab = np.clip(lab, 0, 255)
return cv2.cvtColor(lab.astype(np.uint8), cv2.COLOR_LAB2BGR)

def apply_clahe_mild(img):
    """
    CLAHE with lower clip limit (1.0).
    Provides gentle contrast enhancement.
    """
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=1.0, tileGridSize=(8, 8))
    l = clahe.apply(l)
    lab = cv2.merge((l, a, b))
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

def apply_clahe_very_mild(img):
    """
    Very mild CLAHE (clip limit 0.5).
    Minimal contrast adjustment.
    """
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=0.5, tileGridSize=(8, 8))
    l = clahe.apply(l)
    lab = cv2.merge((l, a, b))
    return cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

def apply_clahe_blended(img, strength=0.5):
    """
    Apply partial CLAHE and blend with original.

    strength=0.5 means:
    50% enhanced image + 50% original image.
    """
    enhanced = apply_clahe_mild(img)
    blended = cv2.addWeighted(img, 1 - strength, enhanced, strength, 0)
    return blended

def apply_white_balance_mild(img, strength=0.3):
    """

```

Apply partial white balance and blend with original.

strength=0.3 means:
30% white-balanced image + 70% original.
"""

```
wb = apply_white_balance_simple(img)
blended = cv2.addWeighted(img, 1 - strength, wb, strength, 0)
return blended
```

```
def apply_white_balance_very_mild(img, strength=0.15):
    """
```

Very subtle white balance correction.

strength=0.15 means:
15% corrected image + 85% original.
"""

```
wb = apply_white_balance_simple(img)
blended = cv2.addWeighted(img, 1 - strength, wb, strength, 0)
return blended
```

```
def apply_bilateral_filter(img, d=9, sigma_color=75, sigma_space=75):
    """
```

Edge-preserving noise reduction.

Smooths homogeneous regions while maintaining sharp edges.
"""

```
return cv2.bilateralFilter(img, d, sigma_color, sigma_space)
```

```
def apply_non_local_means(img, h=10, template_window=7, search_window=21):
    """
```

Non-local means denoising.

State-of-the-art noise reduction that averages
similar patches across the image.
"""

```
return cv2.fastNlMeansDenoisingColored(img, None, h, h, template_window,
search_window)
```

```
def apply_skin_tone_shift(img, shift_amount=20):
    """
```

Shift skin tone by adjusting color channels.
Simulates how same lesion might look on different skin.
"""

```
hsv = cv2.cvtColor(img, cv2.COLOR_RGB2HSV).astype(np.float32)
```

```
# Shift hue (skin tone)
```

```
hsv[:, :, 0] = (hsv[:, :, 0] + shift_amount) % 180
```

```

# Optionally adjust value (brightness)
hsv[:, :, 2] = np.clip(hsv[:, :, 2] * 0.9, 0, 255)

return cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2RGB)

# -----
# Define pairs (DeepDerm + HAM10000)
# -----
pairs = [
]

triples = [

    # DeepDerm strongest theoretical stack
    ("non_local_means", "msrcr", "adaptive_gamma"),

    # Balanced strong fairness stack
    ("bilateral", "msrcr", "local_contrast"),

    # HAM10000-friendly stack
    ("non_local_means", "illumination_comp", "clahe"),

    # Statistical + Retinex hybrid
    ("bilateral", "msrcr", "Z_score_norm"),

    # Conservative fairness pipeline
    ("non_local_means", "illumination_comp", "percentile_norm"),
]

# Map method names to functions
method_map = {
    "clahe": apply_clahe,
    "adaptive_gamma": apply_adaptive_gamma,
    "white_balance": apply_white_balance,
    "msrcr": apply_msrcr,
    "homomorphic": apply_homomorphic,
    "percentile_norm": apply_percentile_norm,
    "local_contrast": apply_local_contrast,
    "illumination_comp": apply_illumination_comp,
    "Z_score_norm": apply_z_score_norm,
    "color_constancy": apply_color_constancy,
    "lab_color_norm": apply_lab_color_normalization,
    "clahe_mild": apply_clahe_mild,
    "clahe_very_mild": apply_clahe_very_mild,
    "clahe_blended": apply_clahe_blended,
    "white_balance_mild": apply_white_balance_mild,
    "white_balance_very_mild": apply_white_balance_very_mild,
    "bilateral": apply_bilateral_filter,
    "non_local_means": apply_non_local_means,

```

```

    "skin_tone_shift": apply_skin_tone_shift,
}
# -----
# Multiprocessing worker
# -----

def process_image(args):
    if len(args) == 3:
        img_path, out_dir, func1 = args
        filename = os.path.basename(img_path)
        try:
            img = cv2.imread(img_path)
            if img is None:
                return f"Failed to read {img_path}"
            img = func1(img)
            cv2.imwrite(os.path.join(out_dir, filename), img)
            return None
        except Exception as e:
            return f"{filename}: {str(e)}"

    img_path, output_dirs = args
    filename = os.path.basename(img_path)

    try:
        img = cv2.imread(img_path)
        if img is None:
            return f"Failed to read {img_path}"

        # New preprocessing methods only

        return None
    except Exception as e:
        return f"Error processing {filename}: {str(e)}"

def process_image_pair(args):
    img_path, out_dir, func1, func2 = args
    filename = os.path.basename(img_path)
    try:
        img = cv2.imread(img_path)
        if img is None:
            return f"Failed to read {img_path}"
        img = func1(img)
        img = func2(img)
        cv2.imwrite(os.path.join(out_dir, filename), img)
        return None
    except Exception as e:
        return f"{filename}: {str(e)}"

def process_image_triple(args):
    img_path, out_dir, func1, func2, func3 = args
    filename = os.path.basename(img_path)

```

```

try:
    img = cv2.imread(img_path)
    if img is None:
        return f"Failed to read {img_path}"

    img = func1(img)
    img = func2(img)
    img = func3(img)

    cv2.imwrite(os.path.join(out_dir, filename), img)
    return None

except Exception as e:
    return f"{filename}: {str(e)}"
# -----
# CLI entry
# -----

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data_dir", default="DDI", help="Root DDI directory")
    parser.add_argument("--exts", default="png,jpg,jpeg", help="Comma-separated
image extensions")
    args = parser.parse_args()

    img_dir = os.path.join(args.data_dir, "images")
    if not os.path.exists(img_dir):
        print(f"Error: {img_dir} does not exist.")
        return

    # Define single-method output directories
    output_dirs = {

    }

    for d in output_dirs.values():
        os.makedirs(d, exist_ok=True)

    # Find images
    print("Finding images...")
    exts = [e.strip().lower() for e in args.exts.split(",") if e.strip()]
    image_paths = []
    for ext in exts:
        image_paths.extend(glob.glob(os.path.join(img_dir, f"*.{ext}")))
    image_paths = sorted(set(image_paths))

    print(f"Found {len(image_paths)} images.")
    if len(image_paths) == 0:

```

```

    print("No images found. Exiting.")
    return

# Prepare arguments for multiprocessing
tasks = [(p, output_dirs) for p in image_paths]

n_cores = cpu_count()
print(f"Processing images using {n_cores} cores...")

with Pool(processes=n_cores) as p:
    results = list(tqdm.tqdm(p.imap(process_image, tasks), total=len(tasks)))

# Report errors if any
errors = [r for r in results if isinstance(r, str) and r]
if errors:
    print("\nSome images failed:")
    for e in errors[:50]:
        print(e)
    if len(errors) > 50:
        print(f"... and {len(errors) - 50} more.")

# -----
# Run all pairs
# -----
for m1, m2 in pairs:
    out_dir = os.path.join(args.data_dir, f"images_{m1}_{m2}")
    os.makedirs(out_dir)
    print(f"\nGenerating pair: {m1} -> {m2}")

    func1 = method_map[m1]
    func2 = method_map[m2]
    tasks = [(p, out_dir, func1, func2) for p in image_paths]

    with Pool(processes=n_cores) as p:
        results = list(tqdm.tqdm(p.imap(process_image_pair, tasks),
total=len(tasks)))

    errors = [r for r in results if isinstance(r, str) and r]
    if errors:
        print(f"{len(errors)} images failed for pair {m1}->{m2}")
        for e in errors[:50]:
            print(e)

# -----
# Run all triples
# -----
for m1, m2, m3 in triples:

    out_dir = os.path.join(

```

```

        args.data_dir,
        f"images_{m1}_{m2}_{m3}"
    )
    os.makedirs(out_dir, exist_ok=True)

    print(f"\nGenerating triple: {m1} -> {m2} -> {m3}")

    func1 = method_map[m1]
    func2 = method_map[m2]
    func3 = method_map[m3]

    tasks = [
        (p, out_dir, func1, func2, func3)
        for p in image_paths
    ]

    with Pool(processes=n_cores) as pool:
        results = list(tqdm.tqdm(
            pool.imap(process_image_triple, tasks),
            total=len(tasks)
        ))

    errors = [r for r in results if isinstance(r, str) and r]
    if errors:
        print(f"{len(errors)} images failed for triple {m1}->{m2}->{m3}")
        for e in errors[:50]:
            print(e)

    print("Processing completed.")

if __name__ == "__main__":
    main()

```

```

=====
FILE: inspect_pkl.py
=====
import pandas as pd
import numpy as np

try:
    data = pd.read_pickle('DDI-results/DeepDerm-evaluation.pkl')
    print("Keys:", data.keys())

    if 'web_path' in data:
        print("Type of web_path:", type(data['web_path']))
        if isinstance(data['web_path'], (list, np.ndarray)):
            print("Length of web_path:", len(data['web_path']))
            print("First 3 web_paths:", data['web_path'][:3])
        else:

```

```

        print("web_path content:", data['web_path'])

    if 'predicted_labels' in data:
        vals = np.array(data['predicted_labels'])
        print("Shape of predicted_labels:", vals.shape)
        print("First 3 predicted_labels:", vals[:3])

    if 'images' in data:
        imgs = data['images']
        print("Type of images:", type(imgs))
        print("Length of images:", len(imgs))
        print("First 3 images:", imgs[:3])

    if 'true_labels' in data:
        t_vals = np.array(data['true_labels'])
        print("Shape of true_labels:", t_vals.shape)
        print("First 3 true_labels:", t_vals[:3])
except Exception as e:
    print("Error:", e)

```

```

=====
FILE: inspect_results.py
=====

```

```

"""

```

```

Inspect evaluation results from a .pkl file.
Usage: python inspect_results.py [path_to.pkl]
Default: DDI-results/DeepDerm-evaluation.pkl
"""

```

```

import pickle
import sys
import os

```

```

def main():
    pkl_path = sys.argv[1] if len(sys.argv) > 1 else
"DDI-results/DeepDerm-evaluation.pkl"
    if not os.path.exists(pkl_path):
        print(f"File not found: {pkl_path}")
        sys.exit(1)
    with open(pkl_path, "rb") as f:
        r = pickle.load(f)
    print("=" * 60)
    print("EVALUATION RESULTS")
    print("=" * 60)

```

```

    if hasattr(r, "columns") and hasattr(r, "shape"): # It's likely a DataFrame
        import pandas as pd
        from sklearn.metrics import roc_auc_score
        print(f"\nLoaded DataFrame with shape: {r.shape}")

```

```

print("Columns:", list(r.columns))

if "y_true" in r.columns and "y_score" in r.columns:
    try:
        auc = roc_auc_score(r["y_true"], r["y_score"])
        print(f"\n--- ROC AUC: {auc:.4f} ---")
    except Exception as e:
        print(f"\n--- ROC AUC: Error calculating ({e}) ---")

print("\n--- Sample size ---")
print("Number of samples:", len(r))
print("\n--- Head ---")
print(r.head())

else: # It's a dictionary (original behavior)
    print("\nKeys in results:", list(r.keys()))
    print("\n--- Classification report ---")
    print(r.get("report", "(no report)"))
    print("\n--- ROC AUC ---")
    print(r.get("ROC_AUC", "(no AUC)"))
    print("\n--- Model / threshold ---")
    print("Model:", r.get("model", "?"), "| Threshold:", r.get("threshold",
"?"))
    print("\n--- Sample size ---")
    true = r.get("true_labels")
    pred = r.get("predicted_labels")
    if true is not None:
        print("Number of samples:", len(true))
    if pred is not None:
        print("Predicted scores shape:", pred.shape if hasattr(pred, "shape")
else len(pred))

if __name__ == "__main__":
    main()

```

```

=====
FILE: optimization_results.csv
=====
combo,best_gap,percentile_norm_range_
Z_score_norm+percentile_norm,0.02695952594874429,98.76833933767236
bilateral+Z_score_norm,0.021338724168912915,78.73889375675077
bilateral+msrcr+Z_score_norm,0.02870031660462924,97.29994639258697
illumination_comp,0.0355404740512556,12.633895803134406
illumination_comp+local_contrast,0.01641584306678634,10.318030414994851,1.4959582850
97589
illumination_comp+adaptive_gamma,-0.014568091387498483,16.283237941160298
msrcr+local_contrast,-0.008401788388311293,1.999658717063922
clahe+adaptive_gamma,0.049916035596628605,3.756571026050584

```

```
non_local_means+msrcr,0.025817717023916442,15
non_local_means+msrcr+adaptive_gamma,0.04584616865614155,15
non_local_means+illumination_comp+clahe,0.041284281008000634,15,86.87010715814864,1.
936631475140609
```

```
=====
FILE: regenerate_optimized.py
=====
import os
import glob
import cv2
from multiprocessing.pool import ThreadPool
from multiprocessing import cpu_count
from generate_preprocessed import (
    apply_illumination_comp, apply_bilateral_filter,
    apply_non_local_means, apply_msrcr, apply_adaptive_gamma,
    apply_percentile_norm
)

IMAGE_DIR = os.path.join("DDI", "images")
image_paths = glob.glob(os.path.join(IMAGE_DIR, "*.png"))
n_cores = cpu_count()

combos = [
    ("non_local_means+msrcr", [lambda img: apply_non_local_means(img,
h=14),
lambda img: apply_msrcr(img)]),
    ("bilateral+illumination_comp", [lambda img: apply_bilateral_filter(img,
sigma_color=38.17611636124784),
lambda img: apply_illumination_comp(img, sigma=10.468560122629444)]),
    ("bilateral+msrcr", [lambda img: apply_bilateral_filter(img,
sigma_color=95.41493828699667),
lambda img: apply_msrcr(img)]),
    ("illumination_comp+adaptive_gamma", [lambda img: apply_illumination_comp(img,
sigma=64.49739202603473),
lambda img: apply_adaptive_gamma(img)]),
    ("illumination_comp", [lambda img: apply_illumination_comp(img,
sigma=36.144687286473655)]),
    ("percentile_norm", [lambda img: apply_percentile_norm(img,
range_=80.01213439643897)]),
    ("bilateral", [lambda img: apply_bilateral_filter(img,
sigma_color=57.79324817677015)]),
]

for combo_name, funcs in combos:
    out_dir = os.path.join("DDI", f"images_{combo_name}_optimized")
    os.makedirs(out_dir, exist_ok=True)
    print(f"Generating {combo_name}...")
```

```

def process(img_path):
    filename = os.path.basename(img_path)
    img = cv2.imread(img_path)
    if img is None:
        return f"Failed to read {img_path}"
    for func in funcs:
        img = func(img)
    cv2.imwrite(os.path.join(out_dir, filename), img)

with ThreadPool(processes=n_cores) as pool:
    pool.map(process, image_paths)

print(f" Saved to {out_dir}")

print("Done.")

=====
FILE: verify_methods.py
=====
"""
Verify that each preprocessing method in generate_preprocessed.py
produces output that matches the saved baseline images in DDI/.

Run from your project root:
    python verify_methods.py
"""
import cv2
import numpy as np
import os
from generate_preprocessed import (
    apply_clahe,
    apply_adaptive_gamma,
    apply_white_balance,
    apply_msrrc,
    apply_percentile_norm,
    apply_local_contrast,
    apply_illumination_comp,
    apply_z_score_norm,
    apply_bilateral_filter,
    apply_non_local_means,
)

# Map each method to:
# - the function and its default parameters
# - the folder where baseline images were saved
methods = {
    "clahe": (lambda img: apply_clahe(img),
"images_clahe"),

```

```

    "adaptive_gamma": (lambda img: apply_adaptive_gamma(img),
"images_adaptive_gamma"),
    "white_balance": (lambda img: apply_white_balance(img),
"images_white_balance"),
    "msrcr": (lambda img: apply_msrcr(img),
"images_msrcr"),
    "percentile_norm": (lambda img: apply_percentile_norm(img),
"images_percentile_norm"),
    "local_contrast": (lambda img: apply_local_contrast(img),
"images_local_contrast"),
    "illumination_comp": (lambda img: apply_illumination_comp(img),
"images_illumination_comp"),
    "z_score_norm": (lambda img: apply_z_score_norm(img),
"images_Z_score_norm"),
    "bilateral": (lambda img: apply_bilateral_filter(img),
"images_bilateral"),
    "non_local_means": (lambda img: apply_non_local_means(img),
"images_non_local_means"),
}

```

```

# Use a few sample images for the check

```

```

DDI_DIR = "DDI"

```

```

sample_files = ["000001.png", "000002.png", "000003.png"]

```

```

print("=" * 60)

```

```

print("Verifying preprocessing methods against saved baselines")

```

```

print("=" * 60)

```

```

for method_name, (func, folder) in methods.items():

```

```

    baseline_dir = os.path.join(DDI_DIR, folder)

```

```

    if not os.path.exists(baseline_dir):

```

```

        print(f"\n[SKIP] {method_name}: no baseline folder found at {baseline_dir}")
        continue

```

```

    mismatches = 0

```

```

    missing = 0

```

```

    mean_diffs = []

```

```

    for filename in sample_files:

```

```

        orig_path = os.path.join(DDI_DIR, "images", filename)

```

```

        base_path = os.path.join(baseline_dir, filename)

```

```

        if not os.path.exists(orig_path):
            continue

```

```

        if not os.path.exists(base_path):
            missing += 1
            continue

```

```

        orig = cv2.imread(orig_path)

```

```

base = cv2.imread(base_path)

if orig is None or base is None:
    print(f" Could not read image: {filename}")
    continue

recomputed = func(orig)

if not np.array_equal(recomputed, base):
    mismatches += 1
    mean_diffs.append(abs(recomputed.mean() - base.mean()))

print(f"\n[{'OK' if mismatches == 0 else 'MISMATCH'}] {method_name}")
if mismatches > 0:
    print(f" {mismatches}/{len(sample_files)} images differ")
    print(f" Avg pixel mean difference: {np.mean(mean_diffs):.2f}")
    print(f" --> Function has been modified since baseline was generated")
if missing > 0:
    print(f" {missing} sample files not found in baseline folder")

print("\n" + "=" * 60)
print("Done. Any [MISMATCH] methods need their baseline regenerated")
print("or their function reverted to the original version.")
print("=" * 60)

```

```

=====
FILE: verify_methods.txt
=====
"""

```

Verify that each preprocessing method in generate_preprocessed.py produces output that matches the saved baseline images in DDI/.

Run from your project root:

```
python verify_methods.py
```

"""

```

import cv2
import numpy as np
import os
from generate_preprocessed import (
    apply_clahe,
    apply_adaptive_gamma,
    apply_white_balance,
    apply_msrrc,
    apply_percentile_norm,
    apply_local_contrast,
    apply_illumination_comp,
    apply_z_score_norm,
    apply_bilateral_filter,
    apply_non_local_means,

```

```

)

# Map each method to:
# - the function and its default parameters
# - the folder where baseline images were saved
methods = {
    "clahe": (lambda img: apply_clahe(img),
"images_clahe"),
    "adaptive_gamma": (lambda img: apply_adaptive_gamma(img),
"images_adaptive_gamma"),
    "white_balance": (lambda img: apply_white_balance(img),
"images_white_balance"),
    "msrcr": (lambda img: apply_msrcr(img),
"images_msrcr"),
    "percentile_norm": (lambda img: apply_percentile_norm(img),
"images_percentile_norm"),
    "local_contrast": (lambda img: apply_local_contrast(img),
"images_local_contrast"),
    "illumination_comp": (lambda img: apply_illumination_comp(img),
"images_illumination_comp"),
    "z_score_norm": (lambda img: apply_z_score_norm(img),
"images_Z_score_norm"),
    "bilateral": (lambda img: apply_bilateral_filter(img),
"images_bilateral"),
    "non_local_means": (lambda img: apply_non_local_means(img),
"images_non_local_means"),
}

# Use a few sample images for the check
DDI_DIR = "DDI"
sample_files = ["000001.png", "000002.png", "000003.png"]

print("=" * 60)
print("Verifying preprocessing methods against saved baselines")
print("=" * 60)

for method_name, (func, folder) in methods.items():
    baseline_dir = os.path.join(DDI_DIR, folder)

    if not os.path.exists(baseline_dir):
        print(f"\n[SKIP] {method_name}: no baseline folder found at {baseline_dir}")
        continue

    mismatches = 0
    missing = 0
    mean_diffs = []

    for filename in sample_files:
        orig_path = os.path.join(DDI_DIR, "images", filename)
        base_path = os.path.join(baseline_dir, filename)

```

```

if not os.path.exists(orig_path):
    continue
if not os.path.exists(base_path):
    missing += 1
    continue

orig = cv2.imread(orig_path)
base = cv2.imread(base_path)

if orig is None or base is None:
    print(f" Could not read image: {filename}")
    continue

recomputed = func(orig)

if not np.array_equal(recomputed, base):
    mismatches += 1
    mean_diffs.append(abs(recomputed.mean() - base.mean()))

print(f"\n[{'OK' if mismatches == 0 else 'MISMATCH'}] {method_name}")
if mismatches > 0:
    print(f" {mismatches}/{len(sample_files)} images differ")
    print(f" Avg pixel mean difference: {np.mean(mean_diffs):.2f}")
    print(f" --> Function has been modified since baseline was generated")
if missing > 0:
    print(f" {missing} sample files not found in baseline folder")

print("\n" + "=" * 60)
print("Done. Any [MISMATCH] methods need their baseline regenerated")
print("or their function reverted to the original version.")
print("=" * 60)

```